

INTRODUCTION TO REINFORCEMENT LEARNING - FYS4411

Amund Solum Alsvik & Jørgen Vestly

University of Oslo
Department of Physics

August 2026

FOREWORD

This document is meant to be supplementary to Sutton & Barto and Xiao's books. The main aim is to provide a more concise and approachable manual to reinforcement learning, as the books are somewhat long-winded and unpractical for looking things up. This document includes some examples with corresponding Python snippets, but are mostly focused on the conceptual framework. We recommend also reading the books for extra intuition and more elaborate examples.

The first part of this document is an introduction to reinforcement learning. We begin with introducing the framework for model based reinforcement learning in a probabilistic setting, before we move on to model free reinforcement learning, and different methods to approximate probability distributions. We also discuss deep reinforcement learning as an extension of tabular reinforcement learning. We later include a Bayesian framework for reinforcement learning, and explain the key differences as well as pros and cons of using a Bayesian framework. This order is deliberate, as the Bayesian viewpoint is easier to understand when one has a general intuition of what reinforcement learning is. Hopefully, the Bayesian chapter is detailed enough, so that it is easy to incorporate the various methods introduced in the probabilistic chapters into a Bayesian framework. We end the document with a fun but substantially more advanced bonus example, that ties several of the introduced concepts and methods together. While this example is not "mandatory" it perfectly illustrates the strong points and challenges in reinforcement learning. We highly recommend that you check it out, especially if you are a fan of Pokémon!

Contents

1	FROM NEURAL NETWORKS TO REINFORCEMENT LEARNING	3
1.1	Neural networks as parametrized functions	3
1.2	The fixed-data setting	4
1.3	When outputs become actions	4
1.4	The agent–environment interaction	5

1.5	Markov decision processes	7
1.6	States, actions, rewards, and transition probabilities	7
1.7	Policies	9
1.8	Probabilities of trajectories	10
1.9	Returns and objectives	11
1.10	Value functions	13
1.11	Bellman equations	14
1.12	Optimal value functions	15
1.13	Prediction and control	16
1.14	Dynamic programming	18
1.15	Example of value iteration on FrozenLake	19
1.16	From model-based to model-free reinforcement learning	22
2	LEARNING FROM EXPERIENCE	23
2.1	Sampled experience	23
2.2	Monte Carlo prediction	24
2.3	First-visit and every-visit Monte Carlo	25
2.4	Monte Carlo control	26
2.5	Example of Monte Carlo control on Blackjack	27
2.6	Temporal-difference prediction	30
2.7	SARSA	31
2.8	Q-learning	32
2.9	Example of SARSA and Q-learning on CliffWalking	34
2.10	Comparing Monte Carlo and temporal-difference methods	36
3	FUNCTION APPROXIMATION AND DEEP REINFORCEMENT LEARNING	38
3.1	From tables to parametrized functions	38
3.2	Loss functions and gradient updates	40
3.3	Deep Q-learning	41
3.4	Policy parametrization	43
3.5	Policy gradient methods	44
3.6	Actor-critic methods	46
3.7	Practical issues in deep reinforcement learning	48
3.8	Proximal policy optimization	49
3.9	Example of PPO on CartPole	51
3.10	Summary of function approximation and deep reinforcement learning	53
4	BAYESIAN REINFORCEMENT LEARNING AND UNCERTAINTY	55
4.1	From stochastic dynamics to uncertain models	55
4.2	Bayesian inference for finite transition models	57
4.3	Posterior uncertainty and exploration	59
4.4	Posterior sampling and Thompson sampling	61
4.5	Example of Thompson sampling for a Bernoulli bandit	62

4.6	Exploration as uncertainty reduction	64
4.7	From bandits back to Markov decision processes	65
4.8	Bayesian model-based reinforcement learning	66
4.9	Optimism and upper-confidence planning	68
4.10	Uncertainty in model-free reinforcement learning	70
4.11	Uncertainty in deep reinforcement learning	72
4.12	The Bayes-adaptive viewpoint	74
4.13	Implementation example comparing exploration strategies	75
4.14	Summary of the Bayesian perspective	80

5 BONUS EXAMPLE: AGENT PLAYS POKÉMON 83

1 FROM NEURAL NETWORKS TO REINFORCEMENT LEARNING

Most of the students taking this course has had an introduction to feed forward neural networks through FYS-STK3155/4155 or other courses. We assume familiarity with this concept and use it as a springboard for our journey into the world of reinforcement learning. We also assume familiarity with basic probability theory as well as stochastic methods such as Monte Carlo methods. The most important prerequisite is a good understanding of linear algebra.

1.1 Neural networks as parametrized functions

A neural network is a parametrized function. Abstractly, we may write $f_\theta : X \rightarrow Y$, where X is an input space, Y is an output space, and $\theta \in \Theta$ denotes the collection of trainable parameters. In a standard feedforward neural network, θ consists of the weights and biases appearing in the affine transformations between layers. For example, a simple feedforward network has the form

$$f_\theta(x) = W_L \sigma(W_{L-1} \sigma(\cdots \sigma(W_1 x + b_1) \cdots) + b_{L-1}) + b_L,$$

where W_i are weight matrices, b_i are bias vectors, and σ is a nonlinear activation function. The precise architecture is not important here. What matters is that the network defines a function whose behavior is controlled by the parameters θ .

Training the network means choosing θ so that f_θ behaves well according to some objective. In many familiar machine learning problems, this objective measures how well the network output matches a desired output. For instance, if $f_\theta(x)$ is supposed to approximate some target value y , then one may measure the error using a loss function

$$\ell(f_\theta(x), y).$$

Thus, at the most basic level, ordinary neural network training is an optimization problem over parameters

choose $\theta \in \Theta$ so that f_θ performs well.

The next question is what kind of data and objective are used to decide what “performs well” means. This leads to the fixed-data setting.

1.2 The fixed-data setting

In many introductory machine learning problems, the parameters of the network are trained from a fixed dataset

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N.$$

Here $x_i \in X$ is an input, $y_i \in Y$ is the corresponding target output, and N is the number of training examples. The dataset $\mathcal{D}$ is assumed to be given before training begins. The purpose of training is to choose parameters $\theta \in \Theta$ such that

$$f_\theta(x_i) \approx y_i$$

for the examples in the dataset, and ideally also for new examples not used during training. To make this into an optimization problem, one introduces a loss function

$$\ell : Y \times Y \rightarrow \mathbb{R}.$$

The quantity $\ell(f_\theta(x_i), y_i)$ measures how poorly the network output $f_\theta(x_i)$ matches the desired output y_i on the i -th example. The empirical loss is then defined by

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f_\theta(x_i), y_i).$$

Training the neural network means solving, or approximately solving, the optimization problem

$$\theta^* \in \arg \min_{\theta \in \Theta} L(\theta).$$

This is the standard fixed-data setting. The important point is that the dataset

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$$

does not depend on the current parameters θ . During training, changing θ changes the network outputs $f_\theta(x_i)$, and therefore changes the value of the loss. However, it does not change the inputs x_i , the target outputs y_i , or the distribution from which the training examples were collected. This is the main structural feature we will contrast with reinforcement learning. In the fixed-data setting, the model is trained on data that already exist. In reinforcement learning, the model may choose actions that influence which data will be observed in the future.

1.3 When outputs become actions

The fixed-data setting describes a situation where the output of the network is compared to a target value. Reinforcement learning begins from a different idea. The output of the model may instead be used to choose an action. Instead of using a neural network as a map $f_\theta : X \rightarrow Y$ we

may use a parametrized function

$$\pi_\theta : \mathcal{S} \rightarrow \mathcal{A}.$$

Here $\mathcal{S}$ is a set of possible states and $\mathcal{A}$ is a set of possible actions. The notation π_θ is standard in reinforcement learning, and such a map is called a policy. In the deterministic case, the interpretation is simply

$$A_t = \pi_\theta(S_t).$$

Given the current state S_t , the policy chooses an action A_t . In many reinforcement learning problems, however, the policy is stochastic. Instead of choosing one action directly, the policy assigns probabilities to actions:

$$\pi_\theta(a \mid s) = \mathbb{P}(A_t = a \mid S_t = s).$$

Thus, for each state s , the policy defines a probability distribution over possible actions. This is the first important conceptual shift. In the fixed-data setting, the output $f_\theta(x)$ is evaluated by comparing it to a target y . In reinforcement learning, the output of the model is used to act. The quality of that action is not necessarily known immediately. It is evaluated through what happens afterward. The second important shift is that actions affect future data. If the policy chooses action A_t in state S_t , the next observed state S_{t+1} may depend on that action. Therefore the sequence of data observed during learning depends on the policy itself. Schematically, the fixed-data setting has the form

$$x_i \mapsto f_\theta(x_i) \mapsto \ell(f_\theta(x_i), y_i),$$

where the data (x_i, y_i) are fixed. In reinforcement learning, the basic structure is instead

$$S_t \mapsto A_t \mapsto S_{t+1}.$$

The output of the model is no longer merely an answer to be checked. It is an action that influences the future evolution of the system. This is why reinforcement learning is a theory of sequential decision-making rather than only function approximation. Neural networks may still be used, but they now appear inside a feedback loop.

1.4 The agent–environment interaction

We now place the policy inside a feedback loop. At each discrete time

$$t = 0, 1, 2, \dots,$$

there is a state S_t . The policy uses this state to choose an action A_t . The environment then responds by producing two new quantities, a reward R_{t+1} and a next state S_{t+1} . The basic interaction is therefore

$$S_t \longrightarrow A_t \longrightarrow (R_{t+1}, S_{t+1}).$$

The indexing convention is chosen to emphasize causality. The action A_t is chosen at time t , while the reward R_{t+1} is received after this action has been applied. Thus the transition from time t to time $t + 1$ is described by

$$(S_t, A_t) \mapsto (R_{t+1}, S_{t+1}).$$

A full interaction generates a sequence

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, S_3, \dots$$

This sequence is called a trajectory. If the interaction stops after a finite number of steps, the trajectory is often called an episode. The important new point is that the data are generated recursively. First, the current state S_t determines which action the policy chooses. Then the chosen action influences the next state S_{t+1} . This means that the future inputs to the policy depend on the previous outputs of the policy. In symbols, the policy participates in generating the sequence

$$S_0, S_1, S_2, \dots$$

through its choices

$$A_0, A_1, A_2, \dots$$

The reward sequence

$$R_1, R_2, R_3, \dots$$

is the feedback signal used to evaluate the policy. However, the reward after one step need not be enough to judge whether an action was good. An action may have delayed consequences. For this reason, reinforcement learning is concerned with the total reward accumulated along a trajectory, rather than only with immediate rewards. This gives the informal structure of reinforcement learning:

$$\begin{array}{ll} \text{policy chooses actions} & \longrightarrow \text{environment produces new states and rewards} \\ & \longrightarrow \text{policy is evaluated by accumulated reward.} \end{array}$$

The next step is to make the environment response precise. This leads to the state space, action space, reward space, and the transition probability $p(s', r \mid s, a)$. The Markov assumption is that this conditional distribution depends only on the current state-action pair (S_t, A_t) , and not on the earlier history. More precisely, if

$$H_t = (S_0, A_0, R_1, \dots, S_t, A_t)$$

denotes the history up to time t , then

$$\mathbb{P}(S_{t+1} = s', R_{t+1} = r \mid H_t) = p(s', r \mid S_t, A_t).$$

Equivalently, when $S_t = s$ and $A_t = a$,

$$\mathbb{P}(S_{t+1} = s', R_{t+1} = r \mid H_t) = p(s', r \mid s, a).$$

This gives the Markov property, meaning that the current state-action pair contains all the information from the past needed to describe the one-step distribution of the future.

1.5 Markov decision processes

The objects introduced above are usually collected into a single mathematical model called a Markov decision process (MDP). An MDP is the mathematical description of the environment in which the agent acts. In the finite case, it consists of the state space $\mathcal{S}$, the action space $\mathcal{A}$, the reward space $\mathcal{R} \subseteq \mathbb{R}$, the admissible action sets $\mathcal{A}(s)$, and the transition-reward kernel

$$p(s', r \mid s, a).$$

These objects specify what states the environment can be in, what actions are available to the agent, what rewards can be received, and how the environment responds after an action is chosen. The word Markov refers to the fact that the one-step response of the environment depends only on the current state-action pair (S_t, A_t) , and not on the entire previous history. Thus, once S_t and A_t are known, the distribution of (R_{t+1}, S_{t+1}) is determined by the kernel $p(s', r \mid s, a)$.

A policy is not part of the environment itself. Rather, the Markov decision process describes the world in which the agent acts, while the policy describes how the agent chooses actions in that world. Once an initial distribution μ_0 and a policy π are chosen, the Markov decision process and the policy together determine the probability law of the trajectories generated by the interaction.

1.6 States, actions, rewards, and transition probabilities

We now formalize the objects appearing in the interaction

$$(S_t, A_t) \mapsto (R_{t+1}, S_{t+1}).$$

The purpose of the formalism is to specify what values these random variables may take, and how the environment responds when an action is chosen.

We begin with three sets

$$\mathcal{S}, \quad \mathcal{A}, \quad \mathcal{R} \subseteq \mathbb{R}.$$

The set $\mathcal{S}$ is called the state space, $\mathcal{A}$ is called the action space, and $\mathcal{R}$ is called the reward space. Thus

$$S_t \in \mathcal{S}, \quad A_t \in \mathcal{A}, \quad R_{t+1} \in \mathcal{R}.$$

The reward space is taken to be a subset of $\mathbb{R}$ because rewards are numerical quantities that will later be added together. In general, not every action has to be available in every state.

Therefore, for each state $s \in \mathcal{S}$, we specify a nonempty subset

$$\mathcal{A}(s) \subseteq \mathcal{A}.$$

The set $\mathcal{A}(s)$ is the set of actions available in state s . We say that a pair (s, a) is admissible if

$$s \in \mathcal{S} \quad \text{and} \quad a \in \mathcal{A}(s).$$

Equivalently, the set of admissible state-action pairs is

$$\mathcal{D} = \{(s, a) : s \in \mathcal{S}, a \in \mathcal{A}(s)\} \subseteq \mathcal{S} \times \mathcal{A}.$$

The environment response is described by a transition-reward probability kernel. In the finite case, this may be viewed as a map

$$p : \mathcal{D} \times \mathcal{S} \times \mathcal{R} \rightarrow [0, 1].$$

Instead of writing $p((s, a), s', r)$, we use the standard conditional notation

$$p(s', r \mid s, a).$$

Here $(s, a) \in \mathcal{D}$ is the current admissible state-action pair, while $(s', r) \in \mathcal{S} \times \mathcal{R}$ is a possible next state-reward pair. For each fixed admissible pair $(s, a) \in \mathcal{D}$, the function

$$(s', r) \longmapsto p(s', r \mid s, a)$$

is required to be a probability distribution on $\mathcal{S} \times \mathcal{R}$. That is,

$$p(s', r \mid s, a) \geq 0$$

for all $s' \in \mathcal{S}$ and $r \in \mathcal{R}$, and

$$\sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) = 1.$$

Thus, for each possible current state s and chosen action a , the kernel p gives a probability distribution over the possible next state s' and reward r . In terms of the random variables in the agent–environment interaction, the intended interpretation is

$$p(s', r \mid s, a) = \mathbb{P}(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a).$$

Thus $p(s', r \mid s, a)$ is the probability that the next state is s' and the received reward is r , given that the current state is s and the chosen action is a . Sometimes it is useful to separate the transition part from the reward part. The marginal probability of transitioning to s' is

$$p(s' \mid s, a) = \sum_{r \in \mathcal{R}} p(s', r \mid s, a).$$

Similarly, the expected one-step reward associated with (s, a) is

$$r(s, a) = \sum_{s' \in \mathcal{S}} \sum_{\rho \in \mathcal{R}} \rho p(s', \rho \mid s, a).$$

The data

$$(\mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{A}(\cdot), p)$$

describe the possible states, the possible actions, the possible rewards, and the probabilistic response of the environment. Together with a policy, these objects determine the probability law of the trajectories generated by the agent–environment interaction.

1.7 Policies

We have already introduced the policy as the rule used by the agent to choose actions. We now describe this object in more detail. In the deterministic case, a policy is a function

$$\pi : \mathcal{S} \rightarrow \mathcal{A}$$

such that

$$\pi(s) \in \mathcal{A}(s)$$

for every $s \in \mathcal{S}$. Thus, if the current state is $S_t = s$, the agent chooses

$$A_t = \pi(s).$$

In this case, the same state always produces the same action. On the other hand, a stochastic policy assigns, to each state s , a probability distribution over the admissible actions $\mathcal{A}(s)$. We write this as

$$\pi(a \mid s) = \mathbb{P}(A_t = a \mid S_t = s).$$

For each fixed state s , the map

$$a \mapsto \pi(a \mid s)$$

satisfies

$$\pi(a \mid s) \geq 0$$

for all $a \in \mathcal{A}(s)$, and

$$\sum_{a \in \mathcal{A}(s)} \pi(a \mid s) = 1.$$

We also set

$$\pi(a \mid s) = 0$$

whenever $a \notin \mathcal{A}(s)$. The deterministic case can be viewed as a special case of the stochastic one. If

$$\mu : \mathcal{S} \rightarrow \mathcal{A}$$

is deterministic with $\mu(s) \in \mathcal{A}(s)$, then it corresponds to the stochastic policy

$$\pi(a \mid s) = \begin{cases} 1, & a = \mu(s), \\ 0, & a \neq \mu(s). \end{cases}$$

Thus deterministic policies concentrate all probability on one action, while stochastic policies may assign positive probability to several actions. When a policy is represented by a neural network with parameters θ , we write π_θ . In a deterministic setting, the network outputs the action directly by

$$A_t = \pi_\theta(S_t).$$

In a stochastic setting, the network parametrizes the probabilities

$$\pi_\theta(a \mid s).$$

For example, if the action set is finite, the network may output one real number for each possible action, and a softmax layer may be used to convert these numbers into probabilities, similarly to the procedure from logistic regression, giving something like

$$\pi_\theta(a \mid s) = \frac{\exp(z_\theta(s, a))}{\sum_{b \in \mathcal{A}(s)} \exp(z_\theta(s, b))}.$$

Here $z_\theta(s, a)$ denotes the score, or logit, assigned by the network to action a in state s . The policy is the object through which the agent influences the trajectory. The environment kernel

$$p(s', r \mid s, a)$$

describes what happens after action a is chosen in state s , while the policy

$$\pi(a \mid s)$$

describes how likely the agent is to choose that action. Together, π and p determine the probability law of the interaction. This relationship may be summarized schematically as follows.

$$S_t \xrightarrow{\pi(\cdot \mid S_t)} A_t \xrightarrow{p(\cdot, \cdot \mid S_t, A_t)} (R_{t+1}, S_{t+1})$$

1.8 Probabilities of trajectories

We have now specified the two one-step rules of the interaction. The policy gives

$$\mathbb{P}(A_t = a_t \mid S_t = s_t) = \pi(a_t \mid s_t),$$

while the environment kernel gives

$$\mathbb{P}(S_{t+1} = s_{t+1}, R_{t+1} = r_{t+1} \mid S_t = s_t, A_t = a_t) = p(s_{t+1}, r_{t+1} \mid s_t, a_t).$$

We now use these one-step rules to describe the probability of a full finite trajectory. To start the process, we specify an initial distribution

$$\mu_0 \in \text{Prob}(\mathcal{S}),$$

where

$$\mu_0(s) = \mathbb{P}(S_0 = s).$$

Now consider a finite trajectory

$$\tau = (s_0, a_0, r_1, s_1, a_1, r_2, \dots, s_{T-1}, a_{T-1}, r_T, s_T).$$

The probability of this trajectory is obtained from the chain rule for joint probabilities. In general, for random variables $X_0, \dots, X_n$,

$$\mathbb{P}(X_0 = x_0, \dots, X_n = x_n) = \mathbb{P}(X_0 = x_0) \prod_{k=1}^n \mathbb{P}(X_k = x_k \mid X_0 = x_0, \dots, X_{k-1} = x_{k-1}).$$

Applying this to the sequence

$$S_0, A_0, R_1, S_1, \dots, A_{T-1}, R_T, S_T,$$

and then using the policy rule and the Markov property, gives

$$\mathbb{P}_\pi(\tau) = \mu_0(s_0) \prod_{t=0}^{T-1} \pi(a_t \mid s_t) p(s_{t+1}, r_{t+1} \mid s_t, a_t).$$

The subscript in $\mathbb{P}_\pi$ emphasizes that the probability law depends on the policy. More precisely, the trajectory law is determined by the initial distribution μ_0 , the policy π , and the environment kernel p . This formula is useful because it separates the two sources of randomness. The factors

$$\pi(a_t \mid s_t)$$

come from the agent's action choices, while the factors

$$p(s_{t+1}, r_{t+1} \mid s_t, a_t)$$

come from the environment's response. Changing the policy therefore changes the probability distribution over trajectories.

1.9 Returns and objectives

The reward R_{t+1} gives immediate feedback after the agent chooses action A_t . However as previously mentioned, the goal in reinforcement learning is usually not to maximize only the next reward. An action may have consequences that appear many time steps later. We therefore

need a quantity that measures accumulated future reward. The return from time t is defined by

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots ,$$

or, more compactly,

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

Here

$$\gamma \in [0, 1]$$

is called the discount factor. It determines how strongly future rewards are weighted relative to immediate rewards. If $\gamma = 0$, then

$$G_t = R_{t+1},$$

so only the immediate reward matters. If γ is close to 1, then rewards far in the future still contribute significantly to the return. In an episodic problem, where the interaction terminates after some final time T , the return is instead written as the finite sum

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}.$$

For example,

$$G_0 = R_1 + \gamma R_2 + \gamma^2 R_3 + \cdots + \gamma^{T-1} R_T.$$

The return satisfies the useful recursive identity

$$G_t = R_{t+1} + \gamma G_{t+1}.$$

Indeed,

$$G_{t+1} = R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \cdots ,$$

so multiplying by γ and adding R_{t+1} gives

$$R_{t+1} + \gamma G_{t+1} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = G_t.$$

This simple recursion is the source of the Bellman equations, which will be introduced later. A policy should be evaluated by the returns it produces. If the initial state is sampled from μ_0 , then the performance objective of a policy π is

$$J_{\mu_0}(\pi) = \mathbb{E}_{\pi} [G_0] .$$

Equivalently,

$$J_{\mu_0}(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \right] .$$

The expectation is taken over trajectories generated by the initial distribution μ_0 , the policy π , and the environment kernel p . The notation $\mathbb{E}_{\pi}$ emphasizes the dependence on the policy, but

the environment and initial distribution are also part of the probability law. The reinforcement learning objective is therefore to find a policy with large expected return

$$\pi^* \in \arg \max_{\pi} J_{\mu_0}(\pi).$$

More explicitly,

$$\pi^* \in \arg \max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \right].$$

This is the fundamental optimization problem in reinforcement learning. The policy is not trained to match fixed target labels. Instead, it is trained to generate trajectories with large expected accumulated reward.

1.10 Value functions

The objective

$$J_{\mu_0}(\pi) = \mathbb{E}_{\pi}[G_0]$$

evaluates a policy by averaging the return from the initial distribution μ_0 . This gives one number for the whole policy. In order to make decisions locally, it is useful to ask a more refined question. If the process is currently in a particular state s , how much return should we expect from that point onward?

Fix a policy π . The state-value function of π is defined by

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s].$$

Thus $v_{\pi}(s)$ is the expected return starting from state s , assuming that all future actions are chosen according to the policy π . Using the definition of the return, this can be written as

$$v_{\pi}(s) = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right].$$

The expectation is taken over the future trajectory generated by the policy π and the environment kernel p , conditioned on the event $S_t = s$.

There is also an action-value function. Instead of only conditioning on the current state, we condition on both the current state and the current action, giving us

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a].$$

Thus $q_{\pi}(s, a)$ is the expected return obtained by first taking action a in state s , and then following the policy π afterwards. Equivalently,

$$q_{\pi}(s, a) = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right].$$

The difference between v_{π} and q_{π} is important. The function v_{π} evaluates states, while q_{π}

evaluates state-action pairs, meaning

$$v_\pi(s) = \text{expected return from state } s,$$

whereas

$$q_\pi(s, a) = \text{expected return from choosing action } a \text{ in state } s.$$

Since a stochastic policy chooses actions according to the probabilities $\pi(a \mid s)$, the two value functions are related by

$$v_\pi(s) = \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) q_\pi(s, a).$$

This equation says that the value of a state is the average value of the available actions, weighted by the probabilities with which the policy chooses them. Value functions are central because they turn a long-term objective into quantities attached to individual states and actions. If $q_\pi(s, a)$ is known, then one can compare the long-term consequences of different actions at the same state. This is much more informative than comparing only immediate rewards.

1.11 Bellman equations

The value functions introduced above are not independent of the transition structure of the problem. They satisfy recursive equations, called Bellman equations, which follow from the recursive identity

$$G_t = R_{t+1} + \gamma G_{t+1}.$$

Fix a policy π . Starting from the definition of the state-value function,

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s],$$

and using the recursion for G_t , we get

$$v_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s].$$

To compute this expectation, we first average over the action chosen by the policy, and then over the next state and reward produced by the environment. If the current state is s , the policy chooses action a with probability

$$\pi(a \mid s).$$

After this action is chosen, the environment produces $(S_{t+1}, R_{t+1}) = (s', r)$ with probability

$$p(s', r \mid s, a).$$

Therefore,

$$v_\pi(s) = \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) [r + \gamma v_\pi(s')].$$

This is the Bellman expectation equation for v_π . The same idea gives a recursive equation for the action-value function. Recall that

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a].$$

Since the first action is already fixed to be a , we only need to average over the next state-reward pair, giving us

$$q_\pi(s, a) = \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) [r + \gamma v_\pi(s')].$$

Using the relation

$$v_\pi(s') = \sum_{a' \in \mathcal{A}(s')} \pi(a' \mid s') q_\pi(s', a'),$$

we may also write the Bellman equation for q_π as

$$q_\pi(s, a) = \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) \left[r + \gamma \sum_{a' \in \mathcal{A}(s')} \pi(a' \mid s') q_\pi(s', a') \right].$$

The Bellman equations say that the value of the present is determined by two terms, the expected immediate reward, and the discounted value of the next state. In words,

$$\text{value now} = \text{reward now} + \text{discounted value later}.$$

These equations are central because they replace a long-horizon quantity, the return G_t , by a one-step recursive relation. Most reinforcement learning algorithms can be understood as different ways of estimating or solving these equations.

1.12 Optimal value functions

The value functions v_π and q_π evaluate a fixed policy π . The next question is how to compare different policies. Given two policies π and π' , we say that π' is at least as good as π if

$$v_{\pi'}(s) \geq v_\pi(s)$$

for every state $s \in \mathcal{S}$. Thus, a policy is better if it gives at least as large expected return from every possible starting state. The optimal state-value function is defined by

$$v_*(s) = \sup_{\pi} v_\pi(s).$$

Thus $v_*(s)$ is the best expected return achievable from state s , where the supremum is taken over all policies. Similarly, the optimal action-value function is defined by

$$q_*(s, a) = \sup_{\pi} q_\pi(s, a).$$

Thus $q_*(s, a)$ is the best expected return achievable after first choosing action a in state s , and then behaving optimally afterwards. The optimal value functions satisfy Bellman optimality

equations. For the state-value function, we have

$$v_*(s) = \max_{a \in \mathcal{A}(s)} \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) [r + \gamma v_*(s')].$$

This equation says that if the agent behaves optimally, then in state s it should choose an action a that maximizes the expected immediate reward plus the discounted optimal value of the next state. For the optimal action-value function, the corresponding equation is

$$q_*(s, a) = \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) \left[r + \gamma \max_{a' \in \mathcal{A}(s')} q_*(s', a') \right].$$

Here the first action a is fixed. After the transition to s' , the agent chooses the next action optimally. If q_* is known, then an optimal deterministic policy can be obtained by choosing, in each state,

$$\pi^*(s) \in \arg \max_{a \in \mathcal{A}(s)} q_*(s, a).$$

In words, an optimal policy chooses an action with maximal optimal action-value. This is one reason why action-value functions are especially useful. The function $v_*(s)$ tells us how good the state s is under optimal behavior, while $q_*(s, a)$ directly tells us how good each action a is in that state. Therefore q_* can be used directly to construct a policy.

At this point, the reinforcement learning problem can be summarized as follows. We want to find a policy π with large expected return. Equivalently, we want to estimate value functions well enough to improve the policy. Different reinforcement learning algorithms differ mainly in how they estimate these value functions and how they use them to update the policy.

1.13 Prediction and control

The previous sections introduced value functions for a fixed policy and optimal value functions obtained by comparing over policies. This leads to two central problems in reinforcement learning. The first problem is called prediction, or policy evaluation. Here the policy π is fixed, and the goal is to compute or approximate its value functions:

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s],$$

or

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a].$$

Thus prediction answers the question

How good is this policy?

The second problem is called control. Here the goal is not merely to evaluate a given policy, but to find a better policy, ideally an optimal one. In terms of the optimal action-value function,

this means finding a policy π^* satisfying

$$\pi^*(s) \in \arg \max_{a \in \mathcal{A}(s)} q_*(s, a).$$

Thus control answers the question

What should the agent do?

These two problems are closely related. If we can evaluate a policy, we can often use that evaluation to improve the policy. For example, suppose we know $q_\pi(s, a)$. Then, instead of choosing actions according to π , we may define a new deterministic policy π' by

$$\pi'(s) \in \arg \max_{a \in \mathcal{A}(s)} q_\pi(s, a).$$

This new policy chooses, in each state, an action that has maximal action-value according to the old policy's value function. This idea is called policy improvement. Many reinforcement learning algorithms alternate, either explicitly or implicitly, between these two operations

evaluate the current policy $\longrightarrow$ improve the policy.

This general pattern is called generalized policy iteration. It is one of the central organizing principles of reinforcement learning.

In tabular reinforcement learning, the value functions v_π and q_π may be stored directly as tables. In large or continuous state spaces, this is usually impossible. One then replaces the exact value functions by parametrized approximations, for example

$$v_\pi(s) \approx \hat{v}(s; w),$$

or

$$q_\pi(s, a) \approx \hat{q}(s, a; w),$$

where w denotes trainable parameters. These approximators may be neural networks. Similarly, one may parametrize the policy itself, giving us

$$\pi(a \mid s) \approx \pi_\theta(a \mid s).$$

This is the point where reinforcement learning connects back to neural networks. The network is no longer trained only to match fixed target labels. It is trained either to approximate value functions, to represent a policy, or both. The algorithms introduced later can be organized according to which objects they try to learn, being

value-based methods: learn v_π , q_π , or q_* ,

policy-based methods: learn π_θ directly,

actor–critic methods: learn both a policy and a value function.

In the tabular setting, value functions and policies can be represented directly by tables. Learning then means updating the entries of these tables from Bellman equations or sampled experience. When value functions or policies are represented by neural networks, the table entries are replaced by parametrized functions, and the parameters are adjusted by optimization methods such as stochastic gradient descent. We postpone this function-approximation viewpoint until later.

1.14 Dynamic programming

Dynamic programming refers to methods that use the Bellman equations directly, assuming that the environment model p is known. In this setting, the recursive equations from the previous sections can be turned into explicit algorithms. Suppose first that a policy π is fixed. The Bellman expectation equation says that

$$v_\pi(s) = \sum_{a \in \mathcal{A}(s)} \pi(a | s) \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r | s, a) [r + \gamma v_\pi(s')].$$

This suggests an iterative method for computing v_π . Start with an arbitrary function $v_0 : \mathcal{S} \rightarrow \mathbb{R}$, and define

$$v_{k+1}(s) = \sum_{a \in \mathcal{A}(s)} \pi(a | s) \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r | s, a) [r + \gamma v_k(s')].$$

Under standard assumptions, this sequence converges to v_π . This procedure is called iterative policy evaluation. Once the value function of the current policy has been computed, we can use it to construct an improved policy. Given v_π , define a new policy π' by choosing

$$\pi'(s) \in \arg \max_{a \in \mathcal{A}(s)} \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r | s, a) [r + \gamma v_\pi(s')].$$

The new policy chooses actions greedily with respect to the value function of the old policy. Combining iterative policy evaluation with this improvement step gives policy iteration. Starting from an initial policy π_0 , one alternates between

$$\pi_k \longrightarrow v_{\pi_k} \longrightarrow \pi_{k+1}.$$

The first step evaluates the current policy. The second step improves it. Repeating these two steps produces a sequence of policies that, under standard assumptions, converges to an optimal policy. There is also a more direct method based on the Bellman optimality equation. Start with an arbitrary function $v_0 : \mathcal{S} \rightarrow \mathbb{R}$, and define

$$v_{k+1}(s) = \max_{a \in \mathcal{A}(s)} \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r | s, a) [r + \gamma v_k(s')].$$

This procedure is called value iteration. Instead of fully evaluating one policy before improving it, value iteration repeatedly applies the optimality update itself. Under standard assumptions, v_k converges to the optimal value function v_* . After v_* has been approximated, a policy can be

obtained by choosing actions greedily with respect to v_* , giving us

$$\pi(s) \in \arg \max_{a \in \mathcal{A}(s)} \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) [r + \gamma v_*(s')].$$

1.15 Example of value iteration on FrozenLake

We have now introduced the basic theory of Markov decision processes, policies, returns, value functions, Bellman equations, and dynamic programming. We now turn to a concrete example to show how these concepts are used in an actual computation. The example is FrozenLake-v1, following the model-based dynamic programming case study in Xiao’s reinforcement learning book. The environment is a 4×4 grid,

$$\begin{array}{cccc} S & F & F & F \\ F & H & F & H \\ F & F & F & H \\ H & F & F & G \end{array}$$

where S denotes the start, F denotes a frozen square, H denotes a hole, and G denotes the goal. The agent starts in the upper left corner and tries to reach the goal in the lower right corner. If the agent reaches G , the episode terminates with reward 1. If the agent reaches a hole, the episode terminates with reward 0. If the agent reaches S or F , the episode continues and the reward is 0. The state space consists of the 16 grid positions,

$$\mathcal{S} = \{0, 1, \dots, 15\}.$$

The action space consists of four actions,

$$\mathcal{A} = \{0, 1, 2, 3\}.$$

These actions correspond to moving left, down, right, and up. The environment is slippery, so the direction chosen by the agent is not always the direction in which the agent actually moves. Therefore the transition is stochastic. The model of the environment is stored in `env.P`. For a state s and an action a , the object `env.P[s][a]` is a list of possible outcomes. Each outcome has the form

$$(\text{prob}, \text{next_state}, \text{reward}, \text{terminated}).$$

This is the implementation-level version of the transition-reward kernel

$$p(s', r \mid s, a).$$

The number `prob` is the probability of the transition, `next_state` is the next state s' , `reward` is the reward r , and `terminated` records whether the episode has ended. For example, Xiao gives the transition data for state 14 and action 2 (move right).

$$\text{env.P}[14][2] = [(1/3, 14, 0, \text{False}), (1/3, 15, 1, \text{True}), (1/3, 10, 0, \text{False})].$$

Thus, from state 14, choosing action 2 can keep the agent in state 14 with reward 0, move it to the goal state 15 with reward 1, or move it to state 10 with reward 0. Each of these outcomes has probability 1/3. The Bellman optimality update for the state-value function is

$$v_{k+1}(s) = \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) [r + \gamma v_k(s')].$$

In the implementation, the sum over possible next states and rewards is replaced by a loop over the outcomes stored in `env.P[s][a]`. It is useful to first define a function that computes action values from a given state-value function. This follows Xiao’s helper function `v2q`.

```
def v2q(env, v, state=None, gamma=1.):
    if state is not None:
        q = np.zeros(env.action_space.n)
        for action in range(env.action_space.n):
            for prob, next_state, reward, terminated in env.P[state][action]:
                q[action] += prob * (
                    reward + gamma * v[next_state] * (1. - terminated)
                )
    else:
        q = np.zeros((env.observation_space.n, env.action_space.n))
        for state in range(env.observation_space.n):
            q[state] = v2q(env, v, state, gamma)
    return q
```

Mathematically, this function computes

$$q(s, a) = \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) [r + \gamma v(s')].$$

The factor `1 - terminated` removes the future value term after a terminal transition. This matches the episodic interpretation, since there is no continuation value after the episode has ended. Value iteration repeatedly applies the Bellman optimality update. In Xiao’s implementation, the value function is initialized as a zero vector and then updated until the largest change in one sweep is smaller than a chosen tolerance.

```
def iterate_value(env, gamma=1., tolerant=1e-6):
    v = np.zeros(env.observation_space.n)

    while True:
        delta = 0

        for state in range(env.observation_space.n):
            vmax = max(v2q(env, v, state, gamma))
            delta = max(delta, abs(v[state] - vmax))
            v[state] = vmax
```

```

    if delta < tolerant:
        break

policy = np.zeros((env.observation_space.n, env.action_space.n))

for state in range(env.observation_space.n):
    action = np.argmax(v2q(env, v, state, gamma))
    policy[state][action] = 1.

return policy, v

```

The vector `v` represents the current approximation to the optimal state-value function. The line

```
vmax = max(v2q(env, v, state, gamma))
```

is the Bellman optimality update in code. It computes the value of each possible action from the current state and then keeps the largest one. After the value function has converged, the policy is obtained by choosing an action that maximizes the corresponding action value. The output can be displayed in the original grid shape. Xiao reports the optimal state-value function

$$\begin{bmatrix} 0.8235 & 0.8235 & 0.8235 & 0.8235 \\ 0.8235 & 0 & 0.5294 & 0 \\ 0.8235 & 0.8235 & 0.7647 & 0 \\ 0 & 0.8824 & 0.9412 & 0 \end{bmatrix}$$

and the corresponding optimal policy, with action indices as entries

$$\begin{bmatrix} 0 & 3 & 3 & 3 \\ 0 & 0 & 0 & 0 \\ 3 & 1 & 0 & 0 \\ 0 & 2 & 1 & 0 \end{bmatrix}.$$

This example shows how the abstract objects from the theory appear in a concrete computation. The state space becomes the integers $0, \dots, 15$. The action space becomes the four possible moves. The transition-reward kernel is represented by `env.P`. The value function is represented by a NumPy vector. The Bellman optimality equation becomes a loop over states, actions, and possible transitions. The policy is then obtained by choosing an action that maximizes the computed action value in each state. The important limitation is that this is still a model-based method. The algorithm uses `env.P`, which contains the transition probabilities of the environment. In many reinforcement learning problems, the agent does not know these probabilities. It can observe sampled transitions by interacting with the environment, but it cannot directly loop over all possible outcomes. This is the motivation for model-free reinforcement learning methods.

1.16 From model-based to model-free reinforcement learning

The FrozenLake example illustrates the model-based case. The environment model is available through `env.P`, and value iteration uses this model directly. For each state s and action a , the algorithm can loop over all possible outcomes and their probabilities. Mathematically, this means that the transition-reward kernel

$$p(s', r \mid s, a)$$

is known. This is a strong assumption. In many reinforcement learning problems, the agent does not know the environment kernel. It can choose an action, observe what happens, and store the transition

$$(S_t, A_t, R_{t+1}, S_{t+1}),$$

but it cannot directly access all probabilities of the form

$$p(s', r \mid s, a).$$

Thus, instead of computing exact expectations from the model, the agent must estimate useful quantities from sampled experience. For example, the Bellman optimality update used in value iteration is

$$v_{k+1}(s) = \max_{a \in \mathcal{A}(s)} \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) [r + \gamma v_k(s')].$$

This update requires a sum over all possible next states and rewards, weighted by their transition probabilities. If these probabilities are unknown, this exact computation is no longer available. Model-free reinforcement learning replaces this exact model-based computation by learning from observed transitions. Instead of asking what the environment would do for every possible next state and reward, the agent observes what the environment actually did in sampled interactions. The basic unit of data is therefore no longer the full kernel p , but a transition sample

$$S_t, A_t, R_{t+1}, S_{t+1}.$$

This leads to a new computational problem. The agent must use sampled trajectories to estimate returns, value functions, action-value functions, or policies. Monte Carlo methods do this by averaging returns observed after visiting states or state-action pairs. Temporal-difference methods update value estimates using one-step transitions. Control methods such as SARSA and Q-learning use sampled experience to improve policies without requiring explicit access to the environment model. The conceptual shift from dynamic programming to model-free reinforcement learning is therefore the shift from known probabilities to sampled experience. In dynamic programming, the Bellman equations are used as exact recursive equations involving the model p . In model-free reinforcement learning, the same Bellman structure remains, but the expectations must be estimated from data generated by interaction with the environment.

This completes the first part of the reinforcement learning framework. We have described the agent-environment interaction, formalized it as a Markov decision process, introduced policies,

returns, value functions, Bellman equations, and dynamic programming, and illustrated the model-based case with FrozenLake. The next step is to study how an agent can learn when the model is not given.

2 LEARNING FROM EXPERIENCE

2.1 Sampled experience

In the previous chapter, we described reinforcement learning using a Markov decision process and studied dynamic programming methods for the case where the environment model is known. In that setting, the transition-reward kernel

$$p(s', r \mid s, a)$$

is available, and the agent can compute expectations by summing over all possible next states and rewards. We now move to the more typical reinforcement learning setting where the environment model is not known explicitly. The agent may know the set of possible actions, and it may observe states and rewards, but it does not have direct access to the probabilities governing the environment. Instead, the agent learns by interacting with the environment. At time t , the agent observes a state S_t , chooses an action A_t , receives a reward R_{t+1} , and observes the next state S_{t+1} . The basic unit of experience is therefore the transition sample

$$(S_t, A_t, R_{t+1}, S_{t+1}).$$

This sample contains local information about what happened after one action was taken. It does not reveal the full transition-reward kernel, but it gives one observation drawn from that kernel. A trajectory consists of many such samples,

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, \dots$$

From this sequence, the agent can estimate quantities such as returns, value functions, and action-value functions. The essential idea is to replace exact expectations over the model by empirical estimates obtained from sampled interaction. For example, in dynamic programming, the Bellman expectation equation for a fixed policy is

$$v_\pi(s) = \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) [r + \gamma v_\pi(s')].$$

This equation assumes that the probabilities $p(s', r \mid s, a)$ are known. In model-free reinforcement learning, these probabilities are not known. Instead, the agent observes samples generated by the environment and uses them to update its estimates. The distinction is therefore not a change in the underlying mathematical goal. The agent still wants a policy with large expected return. The difference is computational. Dynamic programming computes updates from a known model. Model-free reinforcement learning estimates the relevant quantities from experience. This chapter studies methods that learn from sampled trajectories. We begin with Monte Carlo

methods, which estimate value functions by averaging complete returns observed after visits to states or state-action pairs.

2.2 Monte Carlo prediction

Dynamic programming methods compute value functions by using the environment model p . Monte Carlo methods take a different approach. They estimate value functions from sampled episodes, without requiring explicit knowledge of the transition probabilities. Fix a policy π . The prediction problem is to estimate the value function

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s].$$

The expectation is taken over all future trajectories generated after visiting state s . If the model is known, this expectation can be studied through the Bellman equation. If the model is not known, we can instead estimate the expectation by observing returns from actual episodes. Suppose an episode is generated by following the policy π ,

$$S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_T.$$

For each time t , the return from that time is

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}.$$

If the state s is visited at time t , then G_t is one sampled return starting from s . Repeating this over many episodes gives many observed returns associated with the same state. The Monte Carlo idea is to estimate $v_\pi(s)$ by averaging these observed returns. If the state s has been visited $N(s)$ times, and the corresponding observed returns are

$$G^{(1)}(s), G^{(2)}(s), \dots, G^{(N(s))}(s),$$

then the Monte Carlo estimate is

$$V(s) = \frac{1}{N(s)} \sum_{i=1}^{N(s)} G^{(i)}(s).$$

This is simply the sample average of returns observed after visits to state s . The justification is the usual law of large numbers. Since $v_\pi(s)$ is an expected return, repeated sampled returns from state s can be averaged to approximate it. Thus Monte Carlo prediction replaces an exact expectation by an empirical average. The same idea applies to action-value functions. For a fixed policy π ,

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a].$$

If the state-action pair (s, a) is visited many times, and the observed returns after these visits are averaged, we obtain an estimate

$$Q(s, a) \approx q_\pi(s, a).$$

Monte Carlo methods therefore learn from complete sampled episodes. They do not require the transition kernel $p(s', r \mid s, a)$, but they do require returns to be observed. For this reason, the basic Monte Carlo methods are most naturally suited to episodic problems, where an episode eventually terminates and the return can be computed.

2.3 First-visit and every-visit Monte Carlo

Monte Carlo prediction estimates value functions by averaging returns observed after visits to states or state-action pairs. There is one small ambiguity that must be resolved. A state may be visited more than once in the same episode. We must therefore decide which visits contribute returns to the average. Fix a policy π , and suppose an episode has been generated

$$S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_T.$$

For each time $t < T$, the return from time t is

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}.$$

If $S_t = s$, then G_t is a sampled return following a visit to state s . In first-visit Monte Carlo prediction, we use only the first time a state appears in an episode. More precisely, if $S_t = s$, then the return G_t is used only if

$$S_j \neq s$$

for all

$$0 \leq j < t.$$

Thus each state contributes at most one return from each episode. In every-visit Monte Carlo prediction, we use every time a state appears in an episode. If $S_t = s$, then G_t is included in the average, regardless of whether the same state appeared earlier in the episode. Both methods estimate the same value function,

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s].$$

The difference is only in how returns are collected from episodes. First-visit Monte Carlo uses one return per state per episode. Every-visit Monte Carlo may use several returns for the same state from a single episode. The same distinction applies to action-value functions. If we want to estimate

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a],$$

then first-visit Monte Carlo uses the first occurrence of the pair (s, a) in an episode, while every-visit Monte Carlo uses all occurrences of the pair (s, a) . A simple way to write the update is to store all observed returns for each state and average them, giving us

$$V(s) = \frac{1}{N(s)} \sum_{i=1}^{N(s)} G^{(i)}(s).$$

Here $N(s)$ is the number of returns collected for state s . In first-visit Monte Carlo, $N(s)$ counts first visits across episodes. In every-visit Monte Carlo, $N(s)$ counts all visits across episodes. In practice, one usually does not store all previous returns. Instead, the average can be updated incrementally. Suppose $V_n(s)$ is the average after n observed returns for state s , and a new return $G_n(s)$ is observed. Then

$$V_n(s) = V_{n-1}(s) + \frac{1}{n} [G_n(s) - V_{n-1}(s)].$$

This formula updates the estimate by moving it a small step toward the newly observed return. It will be useful later, because many reinforcement learning algorithms have this same general form.

2.4 Monte Carlo control

Monte Carlo prediction estimates the value function of a fixed policy. Monte Carlo control uses the same sampled returns to search for a better policy. The goal is no longer only to estimate v_π or q_π , but to improve the behavior of the agent. For control, action-value functions are especially useful. If the agent knows

$$q_\pi(s, a),$$

then it can compare the long-term effect of different actions in the same state. A natural improvement step is to choose an action with maximal estimated action-value, meaning

$$\pi'(s) \in \arg \max_{a \in \mathcal{A}(s)} Q(s, a).$$

Here $Q(s, a)$ is the current Monte Carlo estimate of $q_\pi(s, a)$. The basic idea is therefore to alternate between generating episodes, estimating action values, and improving the policy. An episode is generated by following the current policy. After the episode is complete, returns are computed. These returns are used to update $Q(s, a)$. The policy is then made greedy with respect to the updated action-value estimates. However, there is an important issue. If the policy always chooses only the action that currently looks best, then some actions may never be tried. The agent may then fail to discover that another action is actually better. For this reason, Monte Carlo control must include exploration. One common way to ensure exploration is to use an ε -greedy policy. Given an action-value estimate Q , the agent usually chooses a greedy action, but with probability ε it chooses a random action. For a finite action set $\mathcal{A}(s)$,

an ε -greedy policy can be written as

$$\pi(a \mid s) = \begin{cases} 1 - \varepsilon + \frac{\varepsilon}{|\mathcal{A}(s)|}, & a \in \arg \max_{b \in \mathcal{A}(s)} Q(s, b), \\ \frac{\varepsilon}{|\mathcal{A}(s)|}, & \text{otherwise,} \end{cases}$$

when there is a unique greedy action. This policy assigns most of the probability to the currently best action, but still gives every action positive probability. A simple Monte Carlo control procedure is then as follows. First initialize $Q(s, a)$ arbitrarily. Then repeat the following steps. Generate an episode using an ε -greedy policy derived from Q . Compute the returns from the episode. Use first-visit or every-visit Monte Carlo updates to improve the estimates $Q(s, a)$. Finally, update the policy so that it remains ε -greedy with respect to the new action-value estimates. In incremental form, if the pair (s, a) has been observed n times and the new observed return is $G_n(s, a)$, the Monte Carlo update is

$$Q_n(s, a) = Q_{n-1}(s, a) + \frac{1}{n} [G_n(s, a) - Q_{n-1}(s, a)].$$

More generally, one often writes the update with a step size $\alpha \in (0, 1]$

$$Q(s, a) \leftarrow Q(s, a) + \alpha [G - Q(s, a)].$$

This has the same interpretation as before. The estimate $Q(s, a)$ is moved toward the observed return G . Monte Carlo control is model-free because it does not require the transition kernel $p(s', r \mid s, a)$. The agent only needs sampled episodes. From these episodes, it estimates action values and improves its policy. The price is that updates are made after observing returns, so the basic Monte Carlo method naturally applies to episodic problems.

2.5 Example of Monte Carlo control on Blackjack

We now illustrate Monte Carlo learning using the Blackjack example from Xiao's reinforcement learning book. This example is useful because Blackjack is naturally episodic. Each game eventually terminates, and the return can be computed from the final outcome of the game. The agent does not need to know the transition probabilities of the card game. It only needs to generate many episodes and average the returns observed after state-action pairs. We briefly recall the rules needed for the example. Blackjack is a card game where the player tries to obtain a hand whose value is as close as possible to 21, without exceeding 21. Number cards have their numerical values, face cards count as 10, and an ace can count either as 1 or as 11. An ace that can be counted as 11 without making the hand exceed 21 is called a usable ace. If the player's sum exceeds 21, the player goes bust and loses immediately. At each decision point, the player chooses between two actions,

$$\mathcal{A} = \{\text{stick}, \text{hit}\}.$$

The action hit means that the player asks for another card. The action stick means that the player stops drawing cards and lets the dealer play. The dealer then follows a fixed rule. In

Xiao’s implementation, the dealer draws cards until the dealer’s sum is at least 17. The final reward is 1 if the player wins, -1 if the player loses, and 0 if the game is a draw. In Xiao’s implementation, the actions are represented by

$$0 = \text{stick}, \quad 1 = \text{hit}.$$

A state is represented by three quantities. The first is the player’s current sum, the second is the dealer’s exposed card, and the third records whether the player has a usable ace. Thus an observation has the form

$$(\text{player_sum}, \text{dealer_card}, \text{usable_ace}).$$

In Xiao’s setup, the discount factor is taken to be

$$\gamma = 1.$$

Since all nonterminal rewards are zero, the final reward of the game is the return. The goal is to learn an action-value function

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a]$$

from sampled games. In code, this is represented by an array Q . For each state-action pair (s, a) , the entry $Q(s, a)$ stores the current estimate of the expected return obtained after choosing action a in state s . Xiao uses an array representation adapted to the Blackjack state variables. Since the player sum ranges from 0 to 21, the dealer card from 1 to 10, and the usable ace variable has two possible values, the policy and action-value arrays have shape

$$(22, 11, 2, 2).$$

The final index corresponds to the two possible actions. The following code is the soft-policy Monte Carlo control update from Xiao, written using the notation of the present discussion.

```
def monte_carlo_with_soft(env, episode_num=500000, epsilon=0.1):
    policy = np.ones((22, 11, 2, 2)) * 0.5
    q = np.zeros_like(policy)
    c = np.zeros_like(policy)

    for _ in range(episode_num):
        state_actions = []
        observation, _ = env.reset()

        while True:
            state = ob2state(observation)
            action = np.random.choice(env.action_space.n, p=policy[state])
            state_actions.append((state, action))
```

```

    observation, reward, terminated, truncated, _ = env.step(action)

    if terminated or truncated:
        break

    g = reward

    for state, action in state_actions:
        c[state][action] += 1.
        q[state][action] += (g - q[state][action]) / c[state][action]

    a = q[state].argmax()
    policy[state] = epsilon / 2.
    policy[state][a] += 1. - epsilon

    return policy, q

```

The list `state_actions` stores the state-action pairs encountered during one episode. After the episode ends, the final reward is stored as `g`. Since $\gamma = 1$ and the intermediate rewards are zero, this final reward is the return associated with each state-action pair visited in that episode. The update

$$q(s, a) \leftarrow q(s, a) + \frac{1}{c(s, a)} [G - q(s, a)]$$

is the incremental sample-average update. Here $c(s, a)$ counts how many times the state-action pair (s, a) has been updated, and G is the observed return from the episode. Thus each new episode moves the estimate $q(s, a)$ toward the newly observed return. The policy is then updated to be soft-greedy with respect to the current action-value estimates. First the greedy action is computed by

$$a^* \in \arg \max_{a \in \mathcal{A}} q(s, a).$$

Then the policy assigns most of its probability to this greedy action, while still assigning positive probability to the other action. Since Blackjack has two actions, Xiao's update first assigns

$$\frac{\varepsilon}{2}$$

to both actions, and then adds

$$1 - \varepsilon$$

to the greedy action. Therefore the greedy action receives probability

$$1 - \varepsilon + \frac{\varepsilon}{2},$$

while the other action receives probability

$$\frac{\varepsilon}{2}.$$

This example shows how Monte Carlo control differs from dynamic programming. In the Frozen-Lake value-iteration example, the algorithm used the environment model directly through `env.P`. In the Blackjack Monte Carlo example, the algorithm does not loop over transition probabilities. Instead, it plays many complete episodes, records the state-action pairs that occurred, observes the final return, and updates the action-value estimates by averaging sampled returns. The essential idea is that the expectation defining $q_\pi(s, a)$ is approximated empirically. The more often a state-action pair is visited, the more returns are collected for that pair, and the more stable the estimate becomes. The policy is improved by making it nearly greedy with respect to these estimates, while the ε -soft update ensures that both actions continue to be explored.

2.6 Temporal-difference prediction

Monte Carlo methods estimate value functions from complete returns. This makes them model-free, but it also means that the update for a state is delayed until the episode has finished. Temporal-difference methods are also model-free, but they update value estimates from individual transitions. Fix a policy π . The prediction problem is still to estimate

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s].$$

Monte Carlo prediction uses the observed return G_t as a target for $V(S_t)$. Temporal-difference prediction instead uses the one-step target

$$R_{t+1} + \gamma V(S_{t+1}).$$

This target combines the reward actually observed after taking one step with the current estimate of the value of the next state. The basic temporal-difference update is

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)],$$

where $\alpha \in (0, 1]$ is a step size. The quantity

$$R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

is called the temporal-difference error. It measures the discrepancy between the current estimate $V(S_t)$ and the one-step target $R_{t+1} + \gamma V(S_{t+1})$. This update has the same general form as the incremental Monte Carlo update,

$$\text{new estimate} = \text{old estimate} + \alpha [\text{target} - \text{old estimate}].$$

The difference is the choice of target. Monte Carlo methods use the complete return G_t . Temporal-difference methods use the one-step target $R_{t+1} + \gamma V(S_{t+1})$. The use of $V(S_{t+1})$

makes temporal-difference learning a bootstrapping method. The estimate of one state is updated using the estimate of another state. This is different from Monte Carlo learning, where the update target is a sampled return computed from the actual rewards observed until the end of the episode. The basic temporal-difference prediction algorithm for a fixed policy π proceeds as follows. Initialize $V(s)$ arbitrarily. Generate experience by following the policy π . Whenever a transition

$$(S_t, A_t, R_{t+1}, S_{t+1})$$

is observed, update $V(S_t)$ using

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)].$$

If S_{t+1} is terminal, the continuation value is taken to be zero. Temporal-difference learning can therefore learn before an episode is complete. This is one of its main advantages over Monte Carlo learning. It also does not require the transition kernel $p(s', r \mid s, a)$, which distinguishes it from dynamic programming. In this sense, temporal-difference learning combines ideas from both Monte Carlo methods and dynamic programming. Like Monte Carlo methods, it learns from sampled experience. Like dynamic programming, it uses a Bellman-style one-step update.

2.7 SARSA

Temporal-difference prediction estimates the value function of a fixed policy. To solve a control problem, we need to use temporal-difference updates to improve the policy. One of the simplest methods for doing this is SARSA. SARSA learns an action-value function $Q(s, a)$. The goal is to approximate

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a],$$

while also improving the policy used to generate experience. The name SARSA comes from the five quantities appearing in one transition,

$$S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}.$$

The update uses the current state-action pair, the observed reward, the next state, and the next action chosen by the current policy. The SARSA target is

$$R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}).$$

This is analogous to the temporal-difference target for state values, but it uses action values instead. The SARSA update is

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)].$$

The quantity

$$R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)$$

is the temporal-difference error for the action-value estimate. SARSA is an on-policy method. This means that the same policy is used both to generate behavior and to define the update target. For example, suppose the agent uses an ε -greedy policy derived from Q . The agent observes S_t , chooses A_t using this ε -greedy policy, observes R_{t+1} and S_{t+1} , and then chooses A_{t+1} using the same ε -greedy policy. The update then uses $Q(S_{t+1}, A_{t+1})$.

A basic SARSA algorithm can be described as follows. Initialize $Q(s, a)$ arbitrarily. For each episode, choose the first action A_0 using the current ε -greedy policy. Then, at each time step, take action A_t , observe R_{t+1} and S_{t+1} , choose A_{t+1} from the current policy, and update $Q(S_t, A_t)$ using the SARSA update. The process continues until the episode terminates. The role of ε -greedy exploration is the same as in Monte Carlo control. If the policy were always greedy, the agent might stop trying actions whose current estimates are poor. By keeping $\varepsilon > 0$, the policy continues to explore, and all actions have some chance of being selected. SARSA differs from Monte Carlo control in when the update is performed. Monte Carlo control waits until an episode is complete and then updates using the observed return. SARSA updates after each transition. It therefore does not need to wait until the end of the episode. SARSA also differs from dynamic programming. Dynamic programming uses the model $p(s', r \mid s, a)$ to compute expectations over all possible next states and rewards. SARSA uses only the sampled transition actually observed. It is therefore a model-free temporal-difference control method.

The important feature of SARSA is that it learns the value of the policy it actually follows. If the policy is ε -greedy, then the update includes the effect of exploratory actions. This will become important when comparing SARSA with Q-learning, which uses a different target.

2.8 Q-learning

SARSA is an on-policy temporal-difference control method. Its update uses the next action A_{t+1} actually chosen by the current behavior policy. Q-learning uses a different target. Instead of using the next action that was actually selected, it uses the best action according to the current action-value estimates. Q-learning learns an action-value function $Q(s, a)$ intended to approximate the optimal action-value function

$$q_*(s, a).$$

Recall that q_* satisfies the Bellman optimality equation

$$q_*(s, a) = \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) \left[r + \gamma \max_{a' \in \mathcal{A}(s')} q_*(s', a') \right].$$

Q-learning replaces the expectation over the environment by sampled transitions. When the agent observes a transition

$$(S_t, A_t, R_{t+1}, S_{t+1}),$$

it updates the estimate $Q(S_t, A_t)$ toward the target

$$R_{t+1} + \gamma \max_{a' \in \mathcal{A}(S_{t+1})} Q(S_{t+1}, a').$$

The Q-learning update is

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_{a' \in \mathcal{A}(S_{t+1})} Q(S_{t+1}, a') - Q(S_t, A_t) \right].$$

If S_{t+1} is terminal, the continuation term is taken to be zero. The quantity inside the square brackets is the temporal-difference error for Q-learning,

$$R_{t+1} + \gamma \max_{a' \in \mathcal{A}(S_{t+1})} Q(S_{t+1}, a') - Q(S_t, A_t).$$

It compares the current estimate $Q(S_t, A_t)$ with a one-step estimate of the return obtained by acting optimally from the next state onward. The key difference between SARSA and Q-learning is the target. SARSA uses

$$R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}),$$

where A_{t+1} is the next action actually chosen by the current policy. Q-learning uses

$$R_{t+1} + \gamma \max_{a' \in \mathcal{A}(S_{t+1})} Q(S_{t+1}, a').$$

Thus SARSA learns the value of the behavior policy, while Q-learning learns toward the greedy policy implied by Q . For this reason, Q-learning is called an off-policy method. The agent may behave according to an exploratory policy, for example an ε -greedy policy, while the update target uses the greedy action in the next state. The behavior policy is used to collect data, but the learned value function is pushed toward the value of an optimal greedy policy.

A basic Q-learning algorithm proceeds as follows. Initialize $Q(s, a)$ arbitrarily. At each time step, choose an action A_t , often using an ε -greedy policy derived from Q . After observing R_{t+1} and S_{t+1} , update $Q(S_t, A_t)$ using the Q-learning update. The next action does not need to be chosen before the update is made, because the target uses the maximum over possible next actions. Q-learning is model-free because it does not require access to the transition kernel $p(s', r \mid s, a)$. It is a temporal-difference method because it updates from one-step transitions and bootstraps from current value estimates. It is a control method because its purpose is to learn an action-value function from which a good policy can be extracted. Once Q has been learned, a greedy policy is obtained by choosing

$$\pi(s) \in \arg \max_{a \in \mathcal{A}(s)} Q(s, a).$$

The next example compares SARSA and Q-learning on the same environment. This comparison is useful because the two methods differ only in the target, but this difference can lead to visibly different behavior.

2.9 Example of SARSA and Q-learning on CliffWalking

We now compare SARSA and Q-learning on the CliffWalking environment from Sutton and Barto's book. This is a standard example because the two algorithms use almost identical information, but their different update targets lead to different behavior. The example also illustrates why it matters whether a temporal-difference method is on-policy or off-policy. The environment is a rectangular grid. The agent starts in the lower left corner and the goal is in the lower right corner. Between the start and the goal there is a cliff. If the agent steps into the cliff, it receives a large negative reward and is sent back to the start. Ordinary moves receive reward -1 , while stepping into the cliff gives reward -100 . The episode ends when the agent reaches the goal. The action space consists of four moves,

$$\mathcal{A} = \{\text{up, right, down, left}\}.$$

The state space consists of the grid positions. The task is to learn a policy that reaches the goal while avoiding the cliff. The key difference between SARSA and Q-learning is the target used in the update. SARSA uses the action actually chosen in the next state

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)].$$

Q-learning instead uses the greedy action value in the next state

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_{a' \in \mathcal{A}(S_{t+1})} Q(S_{t+1}, a') - Q(S_t, A_t) \right].$$

Both algorithms may use an ε -greedy policy to choose actions during learning. The difference is that SARSA updates toward the value of the action actually taken under this exploratory policy, while Q-learning updates toward the value of the greedy action. The following helper function chooses an action using an ε -greedy policy.

```
def epsilon_greedy(Q, state, epsilon):
    if np.random.rand() < epsilon:
        return np.random.randint(Q.shape[1])
    return np.argmax(Q[state])
```

For SARSA, the next action is chosen before the update. This is because the target contains $Q(S_{t+1}, A_{t+1})$.

```
def sarsa(env, episode_num=500, alpha=0.5, gamma=1.0, epsilon=0.1):
    n_states = env.observation_space.n
    n_actions = env.action_space.n
    Q = np.zeros((n_states, n_actions))

    episode_returns = []

    for _ in range(episode_num):
        state, _ = env.reset()
```

```

action = epsilon_greedy(Q, state, epsilon)
total_reward = 0

while True:
    next_state, reward, terminated, truncated, _ = env.step(action)
    total_reward += reward

    if terminated or truncated:
        target = reward
        Q[state, action] += alpha * (target - Q[state, action])
        break

    next_action = epsilon_greedy(Q, next_state, epsilon)
    target = reward + gamma * Q[next_state, next_action]
    Q[state, action] += alpha * (target - Q[state, action])

    state = next_state
    action = next_action

episode_returns.append(total_reward)

return Q, episode_returns

```

For Q-learning, the update does not require the next action chosen by the behavior policy. It uses the maximum action value in the next state.

```

def q_learning(env, episode_num=500, alpha=0.5, gamma=1.0, epsilon=0.1):
    n_states = env.observation_space.n
    n_actions = env.action_space.n
    Q = np.zeros((n_states, n_actions))

    episode_returns = []

    for _ in range(episode_num):
        state, _ = env.reset()
        total_reward = 0

        while True:
            action = epsilon_greedy(Q, state, epsilon)
            next_state, reward, terminated, truncated, _ = env.step(action)
            total_reward += reward

            if terminated or truncated:
                target = reward

```

```

        Q[state, action] += alpha * (target - Q[state, action])
        break

    target = reward + gamma * np.max(Q[next_state])
    Q[state, action] += alpha * (target - Q[state, action])

    state = next_state

episode_returns.append(total_reward)

return Q, episode_returns

```

The two pieces of code are almost identical. Both methods collect sampled transitions from the environment, and both update one entry of the table Q after each transition. The difference is only the target. SARSA uses the value of the next action actually selected by the ε -greedy policy. Q-learning uses the value of the best next action according to the current table. This difference matters in CliffWalking. During learning, an ε -greedy policy sometimes takes exploratory actions. Near the cliff, such exploratory actions are dangerous, because a random move can send the agent into the cliff. SARSA takes this risk into account because its update target uses the action actually selected by the exploratory policy. As a result, SARSA tends to learn a safer path that stays farther from the cliff. Q-learning uses the greedy action in its target, even though the agent may still behave ε -greedily while collecting data. It therefore learns values for the greedy policy rather than for the exploratory behavior policy. In CliffWalking, this often leads Q-learning toward the shortest path along the edge of the cliff. This path is optimal for the greedy policy, but during ε -greedy exploration it can lead to frequent falls. After learning, a greedy policy can be extracted from either action-value table by choosing

$$\pi(s) \in \arg \max_{a \in \mathcal{A}} Q(s, a).$$

The example shows that SARSA and Q-learning are both model-free temporal-difference control methods, but they answer slightly different questions. SARSA evaluates and improves the policy that is actually used to generate behavior. Q-learning learns toward the greedy policy implied by the current action-value estimates.

2.10 Comparing Monte Carlo and temporal-difference methods

We have now introduced the main tabular model-free methods. Monte Carlo methods, SARSA, and Q-learning all learn from sampled experience rather than from an explicit environment model. None of them require direct access to the transition-reward kernel

$$p(s', r \mid s, a).$$

Instead, they use observed transitions and rewards generated by interaction with the environment. The first distinction is between complete-return targets and bootstrapped targets. Monte

Carlo methods wait until an episode has ended and then use the observed return

$$G_t$$

as the target. This gives the update

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [G_t - Q(S_t, A_t)].$$

Temporal-difference methods update after one step. Instead of waiting for the complete return, they use a target involving the observed reward and a current value estimate. For state values, the basic temporal-difference target is

$$R_{t+1} + \gamma V(S_{t+1}).$$

For action values, SARSA uses

$$R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}),$$

while Q-learning uses

$$R_{t+1} + \gamma \max_{a' \in \mathcal{A}(S_{t+1})} Q(S_{t+1}, a').$$

The second distinction is between on-policy and off-policy learning. SARSA is on-policy because its update target uses the next action actually chosen by the current behavior policy. If the agent behaves ε -greedily, then SARSA learns action values that reflect this exploratory behavior. Q-learning is off-policy because its update target uses the greedy action value in the next state, even if the action used to collect data is chosen by an exploratory policy. These differences can be summarized as follows.

Method	Target	Main feature
Monte Carlo	G_t	uses complete returns
SARSA	$R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$	on-policy TD control
Q-learning	$R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a')$	off-policy TD control

This table should not be read as a list of unrelated algorithms. Rather, the methods form a sequence of ideas. Monte Carlo learning replaces model-based expectations by empirical averages of complete returns. Temporal-difference learning goes further by replacing complete returns with one-step bootstrapped targets. SARSA and Q-learning then use these temporal-difference targets to learn action-value functions for control. All methods in this chapter have been described in the tabular setting. This means that the value function is stored explicitly as a table,

$$V(s),$$

or

$$Q(s, a).$$

This is appropriate when the number of states and actions is small. However, many reinforcement learning problems have large, high-dimensional, or continuous state spaces. In such problems,

it is impossible to store a separate value for every state or state-action pair. The next step is therefore to replace tables by parametrized function approximators. Instead of storing $V(s)$ for every state, we may use a function

$$\hat{v}(s; w)$$

with trainable parameters w . Instead of storing $Q(s, a)$ for every state-action pair, we may use

$$\hat{q}(s, a; w).$$

Neural networks provide a flexible way to represent such functions. This brings us back to the starting point of these notes, where neural networks were introduced as parametrized functions. The difference is that the targets are now generated by interaction with an environment rather than by a fixed supervised dataset.

3 FUNCTION APPROXIMATION AND DEEP REINFORCEMENT LEARNING

3.1 From tables to parametrized functions

In the previous chapter, value functions were represented explicitly as tables. For example, in a finite state space, a state-value function can be stored by assigning one number to each state,

$$s \longmapsto V(s).$$

Similarly, an action-value function can be stored by assigning one number to each state-action pair,

$$(s, a) \longmapsto Q(s, a).$$

This representation is simple and useful when the number of states and actions is small. The algorithms from the previous chapter update individual table entries using sampled experience. For example, the Q-learning update changes one entry of the table:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_{a' \in \mathcal{A}(S_{t+1})} Q(S_{t+1}, a') - Q(S_t, A_t) \right].$$

Only the entry corresponding to the observed state-action pair (S_t, A_t) is updated. This tabular representation becomes impossible when the state space is large or continuous. If the state s is an image, a vector of sensor readings, a physical configuration, or a continuous control variable, then we cannot store a separate value for every possible state. Even when the state space is finite, the number of states may be so large that a table is computationally useless. The idea of function approximation is to replace the table by a parametrized function. Instead of storing $V(s)$ for every state s , we introduce a function

$$\hat{v}(s; w),$$

where w denotes trainable parameters. The notation $\hat{v}(s; w)$ means that the function is intended to approximate a true value function, for example $v_\pi(s)$ or $v_*(s)$. Similarly, instead of storing $Q(s, a)$ for every state-action pair, we introduce a parametrized action-value function

$$\hat{q}(s, a; w).$$

The parameter vector w now controls the values assigned to many states or state-action pairs at once. This is the central difference from the tabular setting. In a table, changing $Q(s, a)$ changes only one entry. With a parametrized function, changing w may change the value estimate for many inputs at the same time. This can be beneficial because experience from one state may generalize to similar states. It can also be difficult because an update meant to improve one estimate may accidentally make other estimates worse. Neural networks are one important class of function approximators. A neural network value function has the form

$$\hat{v}(s; w) = f_w(s),$$

where the input is the state and the output is a scalar value estimate. A neural network action-value function may have the form

$$\hat{q}(s, a; w) = f_w(s, a),$$

where both the state and action are given as input. For discrete action spaces, it is also common to use a network that takes the state as input and outputs one value for each action

$$f_w(s) = (\hat{q}(s, a_1; w), \dots, \hat{q}(s, a_m; w)).$$

Here $\mathcal{A} = \{a_1, \dots, a_m\}$. The greedy action is then obtained by choosing the largest output component

$$\pi(s) \in \arg \max_{a \in \mathcal{A}} \hat{q}(s, a; w).$$

The learning problem is no longer to update isolated table entries. Instead, sampled experience is used to adjust the parameters w . The update target may still come from Monte Carlo returns or temporal-difference targets, but the target is now used to define a loss function. For example, a temporal-difference target for a value function is

$$Y_t = R_{t+1} + \gamma \hat{v}(S_{t+1}; w).$$

One may then adjust w so that $\hat{v}(S_t; w)$ is closer to Y_t . A simple squared-error loss is

$$L(w) = \frac{1}{2} [Y_t - \hat{v}(S_t; w)]^2.$$

This is where reinforcement learning reconnects with ordinary neural network training. The network is still a parametrized function, and its parameters are still adjusted by optimization. The difference is that the training targets are not fixed labels from a dataset. They are produced from interaction with the environment and from the agent's own current value estimates. This

makes deep reinforcement learning both powerful and unstable.

3.2 Loss functions and gradient updates

When value functions are represented by tables, learning means changing individual table entries. When value functions are represented by parametrized functions, learning means changing the parameters. We therefore need to turn value-function learning into an optimization problem. Suppose we want to approximate the value function v_π by a parametrized function

$$\hat{v}(s; w),$$

where w denotes the trainable parameters. If we had access to the true value $v_\pi(s)$, a natural squared-error loss would be

$$L(w) = \frac{1}{2} [v_\pi(s) - \hat{v}(s; w)]^2.$$

The gradient of this loss is

$$\nabla_w L(w) = -[v_\pi(s) - \hat{v}(s; w)] \nabla_w \hat{v}(s; w).$$

A gradient descent update would therefore move the parameters in the direction

$$w \leftarrow w + \alpha [v_\pi(s) - \hat{v}(s; w)] \nabla_w \hat{v}(s; w),$$

where $\alpha > 0$ is a step size. In reinforcement learning, the true value $v_\pi(s)$ is usually not known. Instead, we replace it by a target constructed from sampled experience. For Monte Carlo learning, the target is the observed return G_t . This gives the loss

$$L_t(w) = \frac{1}{2} [G_t - \hat{v}(S_t; w)]^2.$$

The corresponding update is

$$w \leftarrow w + \alpha [G_t - \hat{v}(S_t; w)] \nabla_w \hat{v}(S_t; w).$$

This is the function-approximation version of the Monte Carlo update. Instead of moving a table entry toward the observed return, we move the parameters so that the function output at S_t moves toward the observed return. For temporal-difference learning, the target is

$$Y_t = R_{t+1} + \gamma \hat{v}(S_{t+1}; w).$$

The corresponding squared-error loss is

$$L_t(w) = \frac{1}{2} [Y_t - \hat{v}(S_t; w)]^2.$$

This suggests the update

$$w \leftarrow w + \alpha [Y_t - \hat{v}(S_t; w)] \nabla_w \hat{v}(S_t; w).$$

Substituting the target gives

$$w \leftarrow w + \alpha [R_{t+1} + \gamma \hat{v}(S_{t+1}; w) - \hat{v}(S_t; w)] \nabla_w \hat{v}(S_t; w).$$

This is called a semi-gradient update. The reason is that the target Y_t also depends on w , but in the update above we treat the target as fixed when taking the gradient. In other words, the gradient is taken only through the current prediction $\hat{v}(S_t; w)$, not through the target. This is the standard form of temporal-difference learning with function approximation. The same idea applies to action-value functions. Suppose we approximate $q(s, a)$ by

$$\hat{q}(s, a; w).$$

For Q-learning, the temporal-difference target is

$$Y_t = R_{t+1} + \gamma \max_{a' \in \mathcal{A}(S_{t+1})} \hat{q}(S_{t+1}, a'; w).$$

The squared-error loss is

$$L_t(w) = \frac{1}{2} [Y_t - \hat{q}(S_t, A_t; w)]^2.$$

The semi-gradient update is

$$w \leftarrow w + \alpha [Y_t - \hat{q}(S_t, A_t; w)] \nabla_w \hat{q}(S_t, A_t; w).$$

When $\hat{v}$ or $\hat{q}$ is represented by a neural network, the gradients are computed by backpropagation. The reinforcement learning algorithm provides the targets, while ordinary optimization methods adjust the network parameters. Thus the neural network machinery is familiar, but the data and targets are generated by interaction with the environment. This is the main conceptual bridge to deep reinforcement learning. A neural network is still trained by minimizing losses using gradient-based methods, but the loss depends on sampled transitions, bootstrapped targets, and the current behavior of the agent.

3.3 Deep Q-learning

In tabular Q-learning, the action-value function is stored as a table $Q(s, a)$. In large state spaces, this table is replaced by a parametrized function

$$\hat{q}(s, a; w).$$

When $\hat{q}$ is represented by a neural network, the resulting method is called deep Q-learning. The purpose of the network is to approximate the optimal action-value function q_* . For a transition sample

$$(S_t, A_t, R_{t+1}, S_{t+1}),$$

the Q-learning target is

$$Y_t = R_{t+1} + \gamma \max_{a' \in \mathcal{A}(S_{t+1})} \hat{q}(S_{t+1}, a'; w).$$

The prediction is

$$\hat{q}(S_t, A_t; w).$$

The corresponding squared-error loss is

$$L_t(w) = \frac{1}{2} [Y_t - \hat{q}(S_t, A_t; w)]^2.$$

The semi-gradient update is

$$w \leftarrow w + \alpha [Y_t - \hat{q}(S_t, A_t; w)] \nabla_w \hat{q}(S_t, A_t; w).$$

For a finite action space, it is common to use a neural network that takes the state as input and outputs one value for each action. If

$$\mathcal{A} = \{a_1, \dots, a_m\},$$

then the network output has the form

$$f_w(s) = (\hat{q}(s, a_1; w), \dots, \hat{q}(s, a_m; w)).$$

The action-value estimate for a particular action is obtained by selecting the corresponding component of the output vector. The greedy action is chosen by

$$a^* \in \arg \max_{a \in \mathcal{A}} \hat{q}(s, a; w).$$

During training, the agent usually does not act greedily all the time. Instead, an ε -greedy policy is often used. With probability $1 - \varepsilon$, the agent chooses a greedy action. With probability ε , it chooses a random action. This allows the agent to explore while still using the current action-value estimates. There is one important instability in deep Q-learning. The target

$$R_{t+1} + \gamma \max_{a' \in \mathcal{A}(S_{t+1})} \hat{q}(S_{t+1}, a'; w)$$

depends on the same parameters w that are being updated. Thus the target changes as the network changes. This makes the learning problem less stable than ordinary supervised learning, where the labels are fixed. A common stabilization technique is to use a separate target network. We introduce a second parameter vector

$$w^-$$

and define the target by

$$Y_t = R_{t+1} + \gamma \max_{a' \in \mathcal{A}(S_{t+1})} \hat{q}(S_{t+1}, a'; w^-).$$

The parameters w are updated frequently, while w^- is updated more slowly. For example, w^- may be copied from w every fixed number of training steps. This makes the target more stable during optimization. Another common technique is experience replay. Instead of updating the

network only from the most recent transition, the agent stores transitions in a replay buffer,

$$(S_t, A_t, R_{t+1}, S_{t+1}).$$

During training, mini-batches of transitions are sampled from this buffer. This makes the data used for gradient updates less strongly correlated in time and allows the same experience to be reused. With experience replay and a target network, the deep Q-learning loss for a sampled mini-batch $\mathcal{B}$ can be written as

$$L(w) = \frac{1}{|\mathcal{B}|} \sum_{(s,a,r,s') \in \mathcal{B}} \frac{1}{2} [y(r, s') - \hat{q}(s, a; w)]^2,$$

where

$$y(r, s') = r + \gamma \max_{a' \in \mathcal{A}(s')} \hat{q}(s', a'; w^-).$$

If s' is terminal, the continuation term is omitted. Deep Q-learning is therefore the neural network version of Q-learning. The tabular update is replaced by a gradient update on a loss function. The action-value table is replaced by a neural network. The basic Bellman optimality idea remains the same, but the approximation introduces new practical issues, especially instability, exploration, and the need to train from sampled experience.

3.4 Policy parametrization

Value-based methods learn a value function and then derive a policy from it. For example, if an action-value function Q has been learned, a greedy policy is obtained by choosing

$$\pi(s) \in \arg \max_{a \in \mathcal{A}(s)} Q(s, a).$$

This is the viewpoint behind Q-learning and deep Q-learning. The policy is not the primary object being optimized. It is obtained indirectly from the learned action-value function. Policy-based methods take a different approach. Instead of learning a value function first and extracting a policy afterward, we represent the policy itself by a parametrized function. In the stochastic case, this means writing

$$\pi_\theta(a \mid s) = \mathbb{P}_\theta(A_t = a \mid S_t = s),$$

where θ denotes trainable parameters. For each state s , the function $\pi_\theta(\cdot \mid s)$ is a probability distribution over actions. For a finite action space

$$\mathcal{A} = \{a_1, \dots, a_m\},$$

a neural network policy may take the state s as input and output one real number for each action. As we briefly discussed in chapter 1, these numbers are called logits. If the logits are denoted by

$$z_\theta(s, a_1), \dots, z_\theta(s, a_m),$$

then a softmax function can be used to define the action probabilities

$$\pi_{\theta}(a_i | s) = \frac{\exp(z_{\theta}(s, a_i))}{\sum_{j=1}^m \exp(z_{\theta}(s, a_j))}.$$

This construction guarantees that

$$\pi_{\theta}(a_i | s) \geq 0$$

for every action a_i , and

$$\sum_{i=1}^m \pi_{\theta}(a_i | s) = 1.$$

Thus the neural network directly parametrizes a stochastic policy. In continuous action spaces, the policy may instead output the parameters of a probability distribution. For example, the network may output a mean vector and a variance parameter, and the action may be sampled from a Gaussian distribution. In that case, $\pi_{\theta}(a | s)$ is a probability density rather than a probability mass function. The main idea is the same. The policy is represented by a trainable probabilistic rule for choosing actions. The objective is still to maximize expected return. If the initial state distribution is μ_0 , the performance objective can be written as

$$J(\theta) = J_{\mu_0}(\pi_{\theta}) = \mathbb{E}_{\pi_{\theta}} [G_0].$$

Equivalently,

$$J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \right].$$

The dependence on θ is important. Changing θ changes the action probabilities, and changing the action probabilities changes the distribution over trajectories. Thus the objective is an expectation over trajectories generated by the policy being optimized. Policy parametrization gives a direct optimization problem

$$\theta^* \in \arg \max_{\theta} J(\theta).$$

This is different from value-based learning. In value-based methods, the policy is usually obtained by maximizing an estimated action-value function. In policy-based methods, the policy itself is the object being trained. There are several reasons to parametrize the policy directly. Stochastic policies are natural for exploration, because they assign probabilities to actions rather than always choosing a single greedy action. Policy parametrization also applies naturally to continuous action spaces, where taking an $\arg \max$ over actions may be difficult. Finally, direct policy optimization forms the basis for many modern deep reinforcement learning methods, including policy gradient methods and actor-critic methods.

3.5 Policy gradient methods

Policy parametrization turns reinforcement learning into an optimization problem over the policy parameters. The objective is

$$J(\theta) = \mathbb{E}_{\pi_{\theta}} [G_0].$$

A policy gradient method tries to adjust θ directly in order to increase this objective. The difficulty is that the expectation is taken over trajectories generated by the policy itself. Changing θ changes the action probabilities, and therefore changes the probability distribution over trajectories. To make this explicit, let

$$\tau = (s_0, a_0, r_1, s_1, \dots, s_{T-1}, a_{T-1}, r_T, s_T)$$

denote a finite trajectory. The objective may be written as

$$J(\theta) = \sum_{\tau} \mathbb{P}_{\pi_{\theta}}(\tau) G_0(\tau).$$

Here $G_0(\tau)$ is the return obtained along the trajectory τ . Taking the gradient gives

$$\nabla_{\theta} J(\theta) = \sum_{\tau} \nabla_{\theta} \mathbb{P}_{\pi_{\theta}}(\tau) G_0(\tau).$$

The useful identity is

$$\nabla_{\theta} \mathbb{P}_{\pi_{\theta}}(\tau) = \mathbb{P}_{\pi_{\theta}}(\tau) \nabla_{\theta} \log \mathbb{P}_{\pi_{\theta}}(\tau).$$

Using this identity, we get

$$\nabla_{\theta} J(\theta) = \sum_{\tau} \mathbb{P}_{\pi_{\theta}}(\tau) \nabla_{\theta} \log \mathbb{P}_{\pi_{\theta}}(\tau) G_0(\tau).$$

Equivalently,

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \mathbb{P}_{\pi_{\theta}}(\tau) G_0].$$

The trajectory probability contains factors from the initial distribution, the policy, and the environment. The dependence on θ comes only from the policy factors. Therefore

$$\nabla_{\theta} \log \mathbb{P}_{\pi_{\theta}}(\tau) = \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t).$$

The environment probabilities do not need to be differentiated. This is one reason policy gradient methods can be model-free. Substituting this into the gradient expression gives

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) G_0 \right].$$

A more refined version uses the return from time t , rather than the full return from the beginning of the episode, giving us

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) G_t \right].$$

This formula says that actions are reinforced in proportion to the returns that follow them. If an action was sampled and was followed by high return, the update increases the probability of taking similar actions in similar states. If it was followed by low return, the update decreases

that probability. The corresponding stochastic gradient ascent update is

$$\theta \leftarrow \theta + \alpha \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) G_t,$$

where $\alpha > 0$ is a step size. This basic Monte Carlo policy-gradient method is often called REINFORCE. It is common to subtract a baseline from the return:

$$\theta \leftarrow \theta + \alpha \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) [G_t - b(S_t)].$$

The baseline $b(S_t)$ may depend on the state, but not on the action sampled at that time. Subtracting such a baseline does not change the expected policy gradient, but it can reduce variance. A natural choice is to use an estimate of the state-value function,

$$b(S_t) \approx v_{\pi_{\theta}}(S_t).$$

Then the quantity

$$G_t - b(S_t)$$

measures whether the observed return was better or worse than expected from that state. Policy gradient methods are important because they optimize the policy directly. They do not require taking a maximum over actions, and they can be used with stochastic policies and continuous action spaces. Their main weakness is that Monte Carlo gradient estimates can have high variance. Actor-critic methods address this by combining policy gradients with learned value functions.

3.6 Actor-critic methods

Policy gradient methods optimize a parametrized policy directly. In the basic REINFORCE update, the policy is updated using sampled returns,

$$\theta \leftarrow \theta + \alpha \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) G_t.$$

This is conceptually simple, but the return G_t can have high variance because it depends on all rewards observed from time t until the end of the episode. Actor-critic methods reduce this variance by combining a parametrized policy with a learned value function. The policy is called the actor. It chooses actions according to a parametrized distribution

$$\pi_{\theta}(a | s).$$

The value function is called the critic. It evaluates states, actions, or state-action pairs under the current policy. For example, the critic may be a parametrized state-value function

$$\hat{v}(s; w),$$

where w denotes the critic parameters. The actor determines how the agent behaves. The critic estimates how good the current situation is. The actor is updated using feedback from the critic, while the critic is updated using temporal-difference learning. A common actor–critic update is based on the advantage function. Recall that the advantage of action a in state s under policy π is

$$A_\pi(s, a) = q_\pi(s, a) - v_\pi(s).$$

The advantage measures whether action a is better or worse than the average action chosen by the policy in state s . If $A_\pi(s, a) > 0$, then action a is better than expected from state s . If $A_\pi(s, a) < 0$, then it is worse than expected. In practice, the true advantage is usually not known. Actor–critic methods therefore use an estimate. A simple one-step estimate is the temporal-difference error

$$\delta_t = R_{t+1} + \gamma \hat{v}(S_{t+1}; w) - \hat{v}(S_t; w).$$

This quantity compares the observed one-step reward plus estimated next-state value with the current estimate of the value of S_t . It can be interpreted as an estimate of the advantage of the action A_t taken in state S_t . The actor update then takes the form

$$\theta \leftarrow \theta + \alpha_\theta \delta_t \nabla_\theta \log \pi_\theta(A_t | S_t),$$

where $\alpha_\theta > 0$ is the actor step size. If δ_t is positive, the update increases the probability of the action that was taken. If δ_t is negative, the update decreases its probability. The critic is updated by temporal-difference learning. For a state-value critic, the update is

$$w \leftarrow w + \alpha_w \delta_t \nabla_w \hat{v}(S_t; w),$$

where $\alpha_w > 0$ is the critic step size. This moves the value estimate $\hat{v}(S_t; w)$ toward the one-step target

$$R_{t+1} + \gamma \hat{v}(S_{t+1}; w).$$

If S_{t+1} is terminal, the continuation term is omitted. Thus the actor and critic are trained together. The critic learns to estimate values under the current policy. The actor uses the critic’s temporal-difference error to update the policy. This gives a model-free method because the updates use sampled transitions rather than the environment kernel $p(s', r | s, a)$. The relationship to the previous methods is useful to keep in mind. REINFORCE updates the policy using complete returns. Actor–critic methods replace these complete returns by value-based estimates, often using temporal-difference errors. This makes updates possible after individual transitions and usually reduces variance. The price is that the update now depends on the quality of the critic. If the critic is inaccurate, it can give misleading feedback to the actor. Actor–critic methods are important because they combine the strengths of value-based and policy-based reinforcement learning. The policy is optimized directly, which makes the method suitable for stochastic and continuous-action problems. At the same time, the learned value function provides a lower-variance learning signal. Many modern deep reinforcement learning algorithms, including advantage actor–critic and proximal policy optimization, are built on this idea.

3.7 Practical issues in deep reinforcement learning

The update rules introduced above describe the basic mathematical structure of deep reinforcement learning. In practice, however, replacing tables by neural networks introduces several difficulties. These difficulties are not separate from the theory. They arise because the targets, the data distribution, and the policy are all changing during learning. The first issue is that reinforcement learning targets are often nonstationary. In supervised learning, the dataset is usually fixed and each input has a target label that does not change during training. In deep reinforcement learning, the target may depend on the current network itself. For example, in deep Q-learning the target has the form

$$Y_t = R_{t+1} + \gamma \max_{a' \in \mathcal{A}(S_{t+1})} \hat{q}(S_{t+1}, a'; w^-).$$

Even when a target network w^- is used, the target changes over time as the target network is updated. Thus the network is trying to fit targets that are themselves produced by a changing learning process. The second issue is that sampled experience is strongly correlated in time. Consecutive transitions from a trajectory are usually not independent. If an agent moves through nearby states, then the samples

$$(S_t, A_t, R_{t+1}, S_{t+1})$$

and

$$(S_{t+1}, A_{t+1}, R_{t+2}, S_{t+2})$$

are closely related. This can make gradient updates unstable, because a mini-batch of recent transitions may represent only a narrow part of the state space. Experience replay is one way to reduce this problem. By storing transitions and sampling mini-batches from a replay buffer, the agent reuses old experience and reduces the temporal correlation between updates. The third issue is exploration. If the agent always chooses the action that currently looks best, it may never discover better actions. In value-based methods, this is often handled by ϵ -greedy exploration. In policy-gradient methods, exploration can be built into the stochastic policy itself. The policy assigns probabilities to actions, and sampling from the policy naturally produces variation in behavior. However, exploration remains delicate. Too little exploration can lead to poor policies, while too much exploration can prevent the agent from exploiting what it has already learned. The fourth issue is the variance of policy-gradient estimates. The REINFORCE update uses sampled returns,

$$\theta \leftarrow \theta + \alpha \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) G_t.$$

The return G_t can vary greatly from episode to episode, especially in long-horizon problems. This means that the gradient estimate can have high variance. Baselines and critics reduce this variance by replacing raw returns with advantage estimates, but the resulting updates then depend on the quality of the value estimates. The fifth issue is bias introduced by bootstrapping. Temporal-difference methods use targets that contain current value estimates, such as

$$R_{t+1} + \gamma \hat{v}(S_{t+1}; w).$$

This makes learning more sample efficient, because updates can be made after individual transitions. However, it also means that the method learns partly from its own predictions. If these predictions are inaccurate, the target may be biased. This is the basic tradeoff between Monte Carlo methods and temporal-difference methods. Monte Carlo targets are based on complete sampled returns and tend to have high variance. Temporal-difference targets use bootstrapping and may introduce bias, but they often have lower variance and allow earlier updates. The sixth issue is that policy updates can be too aggressive. In policy-gradient methods, changing the parameters θ changes the action probabilities. A large update may produce a policy that is very different from the one that generated the data. This can make learning unstable, because the gradient estimate is based on experience from the old policy, while the updated policy may behave differently. Modern policy-gradient algorithms often try to restrict how much the policy changes during one update. These practical issues motivate many of the design choices in modern deep reinforcement learning. Target networks and replay buffers stabilize value-based methods such as deep Q-learning. Baselines, critics, and advantage estimates stabilize policy-gradient methods. Trust-region and proximal methods control the size of policy updates. The underlying objective is still to maximize expected return, but the algorithms must be designed carefully so that neural network training remains stable.

3.8 Proximal policy optimization

Policy-gradient methods update the parameters of a policy in order to increase expected return. In principle, this can be done by gradient ascent on an objective such as

$$J(\theta) = \mathbb{E}_{\pi_\theta}[G_0].$$

In practice, however, policy updates can be unstable. A large change in the parameters θ can produce a policy that is very different from the policy that generated the data. Proximal policy optimization (PPO) is designed to make policy-gradient updates more stable by limiting how much the new policy is allowed to move away from the old policy during one update. Suppose that experience has been collected using an old policy

$$\pi_{\theta_{\text{old}}}.$$

We want to use this experience to update the policy parameters from θ_{old} to a new value θ . For a sampled state-action pair (S_t, A_t) , we compare the probability assigned by the new policy to the probability assigned by the old policy. This gives the probability ratio

$$r_t(\theta) = \frac{\pi_\theta(A_t | S_t)}{\pi_{\theta_{\text{old}}}(A_t | S_t)}.$$

If $r_t(\theta) > 1$, then the new policy assigns higher probability to the sampled action than the old policy did. If $r_t(\theta) < 1$, then the new policy assigns lower probability to that action. The policy-gradient update should also depend on whether the action was better or worse than expected.

This is measured by an advantage estimate,

$$\hat{A}_t.$$

If $\hat{A}_t > 0$, then the sampled action was better than expected, and the update should increase its probability. If $\hat{A}_t < 0$, then the sampled action was worse than expected, and the update should decrease its probability. A simple policy-gradient surrogate objective is

$$L^{\text{PG}}(\theta) = \mathbb{E}_t \left[r_t(\theta) \hat{A}_t \right].$$

This objective increases the probability of actions with positive advantage and decreases the probability of actions with negative advantage. However, by itself it does not prevent the new policy from moving too far away from the old policy. PPO modifies this objective by clipping the probability ratio. For a small parameter $\epsilon > 0$, define the clipped ratio by

$$\text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon).$$

This restricts the ratio to lie between $1 - \epsilon$ and $1 + \epsilon$. The clipped PPO objective is

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right].$$

The minimum is taken to prevent updates that would improve the surrogate objective only by changing the policy too much. If the new policy already assigns much higher probability to an action with positive advantage, the clipped term stops the objective from rewarding an even larger increase. If the new policy already assigns much lower probability to an action with negative advantage, the clipped term similarly limits the update. The critic is trained at the same time. Typically, the critic is a parametrized value function

$$\hat{v}(s; w),$$

and it is trained to approximate returns or bootstrapped value targets. A common value loss has the form

$$L^{\text{V}}(w) = \frac{1}{2} \mathbb{E}_t \left[\left(\hat{V}_t - \hat{v}(S_t; w) \right)^2 \right],$$

where $\hat{V}_t$ is a target value estimate. The actor update uses the clipped policy objective, while the critic update minimizes the value loss. PPO is therefore an actor–critic method. The actor is the policy π_θ , and the critic is the value function $\hat{v}(s; w)$. The critic estimates how good states are, and the actor uses advantage estimates derived from the critic to update the policy. The clipping mechanism makes the actor update conservative, which helps stabilize training. The central idea can be summarized as follows. Ordinary policy-gradient methods try to increase the probability of actions that led to better-than-expected outcomes. PPO keeps this idea, but prevents each update from changing the policy too aggressively. This makes PPO one of the most widely used deep reinforcement learning algorithms in practice.

3.9 Example of PPO on CartPole

We now illustrate proximal policy optimization using the CartPole environment, following the PPO programming task in Xiao’s book. The purpose of this example is to show how the actor–critic structure and the clipped PPO objective appear in an actual implementation. CartPole is a classical control problem. A pole is attached to a cart that can move horizontally. The goal is to move the cart left or right so that the pole remains balanced. At each time step, the agent observes a state vector describing the physical configuration of the system. This observation consists of four real numbers,

$$s = (x, \dot{x}, \varphi, \dot{\varphi}),$$

where x is the cart position, $\dot{x}$ is the cart velocity, φ is the pole angle, and $\dot{\varphi}$ is the angular velocity of the pole. The action space is discrete and consists of two actions,

$$\mathcal{A} = \{0, 1\}.$$

One action pushes the cart to the left, and the other pushes it to the right. This example is different from the tabular examples studied earlier. The state is no longer one of finitely many grid positions that can conveniently be stored in a table. Instead, the state is a vector in a continuous state space. We therefore use neural networks to represent the policy and the value function. The policy network is the actor. It takes the state s as input and outputs a probability distribution over the two actions

$$\pi_{\theta}(a \mid s).$$

For CartPole, this can be implemented by a neural network whose final layer outputs two logits. A softmax function converts the logits into action probabilities. The value network is the critic. It also takes the state s as input, but it outputs a single scalar

$$\hat{v}(s; w).$$

This scalar estimates the expected return from state s under the current policy. During data collection, PPO runs the current policy in the environment and stores the sampled experience. At each time step, the algorithm stores the state, the action sampled from the policy, the reward, the next state, whether the episode terminated, and the probability or log-probability assigned to the sampled action by the old policy. Schematically, one stores samples of the form

$$(S_t, A_t, R_{t+1}, S_{t+1}, \log \pi_{\theta_{\text{old}}}(A_t \mid S_t)).$$

The old log-probability is needed because PPO compares the new policy to the policy that generated the data. After collecting a batch of experience, the critic is used to estimate advantages. A simple one-step advantage estimate is the temporal-difference error

$$\hat{A}_t = R_{t+1} + \gamma \hat{v}(S_{t+1}; w) - \hat{v}(S_t; w).$$

More sophisticated implementations often use multi-step advantage estimates, but the role of the advantage is the same. It measures whether the sampled action was better or worse than expected. The key PPO quantity is the probability ratio

$$r_t(\theta) = \frac{\pi_\theta(A_t | S_t)}{\pi_{\theta_{\text{old}}}(A_t | S_t)}.$$

In code, it is common to compute this ratio using log-probabilities

$$r_t(\theta) = \exp(\log \pi_\theta(A_t | S_t) - \log \pi_{\theta_{\text{old}}}(A_t | S_t)).$$

This is numerically convenient and matches the quantities stored during data collection. The clipped PPO actor objective is

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right].$$

Since most optimization libraries minimize losses rather than maximize objectives, the actor loss is usually implemented as the negative of this expression

$$L_{\text{actor}}(\theta) = -\mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right].$$

The critic is trained by minimizing a squared-error loss. If $\hat{V}_t$ denotes a target return or value target, then

$$L_{\text{critic}}(w) = \frac{1}{2} \mathbb{E}_t \left[\left(\hat{V}_t - \hat{v}(S_t; w) \right)^2 \right].$$

Thus the actor learns how to choose actions, while the critic learns to predict returns. A compact PyTorch-style training step has the following structure.

```
log_probs = actor.log_prob(states, actions)
ratios = torch.exp(log_probs - old_log_probs)

unclipped = ratios * advantages
clipped = torch.clamp(ratios, 1 - epsilon, 1 + epsilon) * advantages

actor_loss = -torch.mean(torch.min(unclipped, clipped))
critic_loss = 0.5 * torch.mean((returns - critic(states)) ** 2)

loss = actor_loss + critic_loss

optimizer.zero_grad()
loss.backward()
optimizer.step()
```

The first line computes the log-probabilities of the sampled actions under the current policy. The second line computes the probability ratio between the current policy and the old policy. The next two lines compute the unclipped and clipped surrogate objectives. Taking the minimum

implements the conservative PPO update. The actor loss updates the policy, and the critic loss updates the value function. The important point is that PPO is an on-policy method. The stored data were generated by $\pi_{\theta_{\text{old}}}$, and the update is designed to avoid moving too far away from that policy. After one or more optimization steps, new experience is collected using the updated policy, and the process repeats. This example shows how several earlier ideas fit together. The policy is represented by a neural network actor. The value function is represented by a neural network critic. Sampled trajectories provide the training data. Advantage estimates tell the actor which actions were better or worse than expected. The probability ratio compares the new policy to the old policy. The clipping mechanism prevents overly large policy updates. Thus PPO combines policy gradients, actor-critic learning, and practical stabilization into a single deep reinforcement learning algorithm.

3.10 Summary of function approximation and deep reinforcement learning

This chapter extended the tabular reinforcement learning methods from the previous chapter to settings where value functions and policies cannot be stored explicitly as tables. In the tabular setting, learning means updating entries such as

$$V(s)$$

or

$$Q(s, a).$$

This is only practical when the state and action spaces are small. In large, high-dimensional, or continuous state spaces, we instead represent value functions and policies by parametrized functions. The basic replacement is

$$V(s) \rightsquigarrow \hat{v}(s; w),$$

and

$$Q(s, a) \rightsquigarrow \hat{q}(s, a; w).$$

Here w denotes trainable parameters. When these functions are represented by neural networks, the resulting methods belong to deep reinforcement learning. The central idea is that sampled experience is used to define targets and losses, and gradient-based optimization is then used to adjust the parameters. For value-function approximation, the update target may come from Monte Carlo returns or temporal-difference learning. A Monte Carlo target uses the complete return

$$G_t,$$

while a temporal-difference target uses a bootstrapped estimate such as

$$R_{t+1} + \gamma \hat{v}(S_{t+1}; w).$$

The corresponding loss compares the network prediction at the current state with the chosen target. For example,

$$L_t(w) = \frac{1}{2} [Y_t - \hat{v}(S_t; w)]^2.$$

This connects reinforcement learning to ordinary neural network training, but with an important difference. The targets are not fixed labels from a supervised dataset. They are generated from interaction with the environment and may depend on the current value estimates. Deep Q-learning applies this idea to action-value functions. The tabular Q -table is replaced by a neural network, and the Q-learning target becomes

$$Y_t = R_{t+1} + \gamma \max_{a' \in \mathcal{A}(S_{t+1})} \hat{q}(S_{t+1}, a'; w^-).$$

The target network w^- and experience replay are practical tools used to make training more stable. The target network slows down changes in the target, while experience replay reduces temporal correlations in sampled data. Policy-based methods take a different route. Instead of learning a value function and deriving a policy afterward, they parametrize the policy directly by

$$\pi_\theta(a \mid s).$$

The objective is to choose parameters θ that maximize expected return,

$$J(\theta) = \mathbb{E}_{\pi_\theta}[G_0].$$

Policy gradient methods estimate $\nabla_\theta J(\theta)$ from sampled trajectories. The basic REINFORCE update has the form

$$\theta \leftarrow \theta + \alpha \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(A_t \mid S_t) G_t.$$

This directly increases the probability of actions that were followed by high return and decreases the probability of actions that were followed by low return. Actor-critic methods combine policy optimization with value-function learning. The actor is the parametrized policy

$$\pi_\theta(a \mid s),$$

and the critic is a value function such as

$$\hat{v}(s; w).$$

The critic provides a learning signal for the actor, often through an advantage estimate or temporal-difference error. A simple actor update is

$$\theta \leftarrow \theta + \alpha_\theta \delta_t \nabla_\theta \log \pi_\theta(A_t \mid S_t),$$

where

$$\delta_t = R_{t+1} + \gamma \hat{v}(S_{t+1}; w) - \hat{v}(S_t; w).$$

The critic is trained to make this value estimate more accurate, while the actor is trained to choose actions with positive estimated advantage. Proximal policy optimization is a modern actor–critic method designed to make policy-gradient updates more stable. It compares the new policy with the old policy through the probability ratio

$$r_t(\theta) = \frac{\pi_\theta(A_t \mid S_t)}{\pi_{\theta_{\text{old}}}(A_t \mid S_t)}.$$

The clipped PPO objective limits the effect of updates that move the new policy too far away from the policy that generated the data. This makes PPO more robust in practice than unconstrained policy-gradient updates. The main lesson of this chapter is that deep reinforcement learning is not a completely separate framework. It is the same reinforcement learning structure developed earlier, but with tables replaced by parametrized functions. Bellman targets, returns, advantages, policies, and value functions remain the central objects. Neural networks provide flexible approximators, while gradient-based optimization provides the mechanism for fitting them. The benefit of function approximation is generalization. Experience from one state can influence predictions and actions in related states. This is essential in large or continuous problems. The cost is instability. Targets can move during training, samples can be correlated, policies can change too quickly, and bootstrapped estimates can propagate errors. Much of modern deep reinforcement learning is concerned with controlling these sources of instability. At this point, we have developed the standard path from tabular reinforcement learning to deep reinforcement learning. The next natural step is to examine reinforcement learning from a Bayesian perspective. In the chapters so far, uncertainty has mostly appeared through stochastic policies, transition kernels, and sampled trajectories. A Bayesian treatment adds another layer by placing probability distributions over unknown quantities such as models, parameters, value functions, or policies. This gives a principled way to reason about uncertainty, exploration, and learning from limited data.

4 BAYESIAN REINFORCEMENT LEARNING AND UNCERTAINTY

4.1 From stochastic dynamics to uncertain models

In the previous chapters, reinforcement learning was already described using probability. The environment was represented by a transition-reward kernel

$$p(s', r \mid s, a),$$

and a stochastic policy was represented by probabilities

$$\pi(a \mid s).$$

Together with an initial distribution μ_0 , these objects determined a probability law over trajectories. In that framework, the environment may be stochastic because the same state-action pair (s, a) may lead to different next states and rewards. This is not yet a Bayesian formulation. The reason is that the transition-reward kernel p was treated as fixed. In dynamic program-

ming, p was assumed to be known. In model-free reinforcement learning, p was unknown, but the agent did not keep an explicit probability distribution over possible choices of p . The agent updated value estimates, action-value estimates, or policy parameters from sampled experience. A Bayesian formulation introduces probability distributions over unknown quantities. We begin with a discrete model class. Let

$$\mathfrak{M} = \{M_1, M_2, M_3, \dots\}$$

be a finite or countable set of possible environment models. Each M_i should be thought of as one possible description of the environment. For example, M_i may specify a particular transition model, a particular reward model, or a full Markov decision process. The Bayesian agent does not initially know which model in $\mathfrak{M}$ is the true one. Before observing data, the agent assigns probabilities to the possible models. This probability distribution is called the prior distribution. For each $M_i \in \mathfrak{M}$, we write

$$\mathbb{P}(M_i) = \mathbb{P}(\text{the true environment model is } M_i).$$

Thus $\mathbb{P}(M_i)$ measures how plausible model M_i is before the observed data are taken into account. The agent then observes data generated by interaction with the environment. After t time steps, the observed data may be written as

$$\mathcal{D}_t = \{(S_k, A_k, R_{k+1}, S_{k+1})\}_{k=0}^{t-1}.$$

Thus $\mathcal{D}_t$ is the collection of transitions and rewards observed so far. For each possible model M_i , we can ask how likely it is that this model would have produced the observed data $\mathcal{D}_t$. This quantity is called the likelihood and is denoted by

$$\mathbb{P}(\mathcal{D}_t \mid M_i).$$

The likelihood measures how compatible the model M_i is with the observed data. As a function of M_i , the likelihood tells us how well each possible model explains the data, but it is not by itself the updated belief distribution over models. After observing $\mathcal{D}_t$, the agent updates its belief about which model is true. The updated distribution is called the posterior distribution. For each $M_i \in \mathfrak{M}$, it is denoted by

$$\mathbb{P}(M_i \mid \mathcal{D}_t).$$

The posterior represents how plausible model M_i is after the observed data have been taken into account. Bayes' rule gives the relation between these quantities:

$$\mathbb{P}(M_i \mid \mathcal{D}_t) = \frac{\mathbb{P}(\mathcal{D}_t \mid M_i)\mathbb{P}(M_i)}{\mathbb{P}(\mathcal{D}_t)}.$$

The numerator contains two factors. The factor $\mathbb{P}(M_i)$ is the prior plausibility of model M_i . The factor $\mathbb{P}(\mathcal{D}_t \mid M_i)$ is the likelihood of the observed data under model M_i . Thus a model receives high posterior probability when it was plausible before seeing the data and also explains

the observed data well. The denominator is the marginal probability of the observed data,

$$\mathbb{P}(\mathcal{D}_t) = \sum_j \mathbb{P}(\mathcal{D}_t \mid M_j) \mathbb{P}(M_j).$$

Its role is to normalize the posterior so that

$$\sum_i \mathbb{P}(M_i \mid \mathcal{D}_t) = 1.$$

Thus Bayes' rule turns prior beliefs and observed data into posterior beliefs. In a continuous model class, the same idea applies, but probabilities assigned to individual models are replaced by probability densities. One may then write a prior density $p(M)$, a likelihood $p(\mathcal{D}_t \mid M)$, and a posterior density $p(M \mid \mathcal{D}_t)$. The normalizing denominator is then written as an integral,

$$p(\mathcal{D}_t) = \int p(\mathcal{D}_t \mid M) p(M) dM.$$

For the present discussion, the discrete notation is easier to interpret, and it contains the essential idea. In reinforcement learning, the most important point is that the posterior is a distribution over possible environments or parameters, not just a single estimate. If several models are still plausible after observing data, then the posterior assigns nonzero probability to several models. This gives the agent a mathematical representation of uncertainty. This should be distinguished from the initial state distribution μ_0 . The distribution μ_0 describes how the first state S_0 is sampled at the beginning of an episode. It is part of the generative description of trajectories. A Bayesian prior over $\mathfrak{M}$, on the other hand, describes uncertainty about which environment model is the true one. These are different kinds of probability distributions. The Bayesian perspective therefore adds an additional probabilistic layer. In the earlier chapters, once p , π , and μ_0 were fixed, they determined a probability law over trajectories. In Bayesian reinforcement learning, the agent may also be uncertain about p or other components of the environment. This uncertainty is represented by a prior distribution, updated through observed transitions, and summarized by a posterior distribution. The next sections make this more concrete by considering finite transition models, where the unknown probabilities can be updated explicitly from observed counts.

4.2 Bayesian inference for finite transition models

We now specialize the Bayesian idea to a finite Markov decision process. The goal is to make the posterior update explicit. To keep the notation simple, we focus first on the transition probabilities and ignore rewards. Thus, for each admissible state-action pair (s, a) , we consider the transition probabilities

$$p(s' \mid s, a), \quad s' \in \mathcal{S}.$$

For fixed (s, a) , these probabilities define a probability distribution over next states by

$$p(s' \mid s, a) \geq 0,$$

and

$$\sum_{s' \in \mathcal{S}} p(s' \mid s, a) = 1.$$

In the Bayesian view, these transition probabilities may be unknown. The agent therefore does not maintain only a single estimate of $p(\cdot \mid s, a)$. Instead, it maintains a probability distribution over possible transition probability vectors. For a fixed pair (s, a) , define

$$\theta_{s,a} = (p(s_1 \mid s, a), p(s_2 \mid s, a), \dots, p(s_n \mid s, a)),$$

where

$$\mathcal{S} = \{s_1, \dots, s_n\}.$$

The vector $\theta_{s,a}$ belongs to the probability simplex,

$$\Delta_n = \left\{ x \in \mathbb{R}^n : x_i \geq 0, \sum_{i=1}^n x_i = 1 \right\}.$$

Thus $\theta_{s,a}$ is the unknown transition distribution from the state-action pair (s, a) . A standard prior distribution on probability vectors is the Dirichlet distribution. We write

$$\theta_{s,a} \sim \text{Dir}(\alpha_1, \dots, \alpha_n),$$

where the parameters satisfy

$$\alpha_i > 0.$$

The parameters α_i may be interpreted as prior pseudo-counts. A larger value of α_i means that, before observing data, the agent assigns more prior weight to transitions into state s_i . Suppose now that the agent observes transitions from the same state-action pair (s, a) . Let

$$N_i(s, a)$$

be the number of observed transitions from (s, a) into state s_i . Thus

$$N_i(s, a) = \#\{k : S_k = s, A_k = a, S_{k+1} = s_i\}.$$

The vector of observed counts is

$$N(s, a) = (N_1(s, a), \dots, N_n(s, a)).$$

The Dirichlet distribution is conjugate to the categorical distribution. This means that if the prior is Dirichlet and the data consist of observed next-state counts, then the posterior is also Dirichlet. More precisely, if

$$\theta_{s,a} \sim \text{Dir}(\alpha_1, \dots, \alpha_n)$$

before observing data, then after observing counts $N_1(s, a), \dots, N_n(s, a)$, the posterior is

$$\theta_{s,a} \mid \mathcal{D}_t \sim \text{Dir}(\alpha_1 + N_1(s, a), \dots, \alpha_n + N_n(s, a)).$$

Thus the Bayesian update has a simple form. The prior pseudo-counts are increased by the observed transition counts. The posterior mean gives a natural point estimate of the transition probability

$$\mathbb{E}[p(s_i | s, a) | \mathcal{D}_t] = \frac{\alpha_i + N_i(s, a)}{\sum_{j=1}^n \alpha_j + \sum_{j=1}^n N_j(s, a)}.$$

This estimate is similar to the empirical frequency

$$\frac{N_i(s, a)}{\sum_{j=1}^n N_j(s, a)},$$

but it also includes the prior pseudo-counts. When little data have been observed, the prior has a strong influence. As more data are collected, the observed counts dominate. This gives a concrete Bayesian interpretation of learning a transition model. Standard frequency estimation keeps track of counts and forms a point estimate. Bayesian inference keeps track of a full posterior distribution over transition probabilities. The posterior mean gives one possible estimate, but the posterior also describes uncertainty. If only a few transitions have been observed from (s, a) , the posterior distribution is broad. If many transitions have been observed, the posterior becomes more concentrated. The same construction can be applied independently to each state-action pair. For each admissible (s, a) , the agent maintains a Dirichlet posterior over the possible next-state distribution. The collection of these posteriors represents uncertainty about the transition model of the Markov decision process. This is the simplest example of Bayesian reinforcement learning. The agent begins with prior beliefs about the transition probabilities, observes transitions through interaction, and updates its posterior beliefs by adding observed counts. The next question is how such posterior uncertainty can be used for decision-making and exploration.

4.3 Posterior uncertainty and exploration

The previous subsection described how an agent can maintain posterior beliefs about transition probabilities in a finite Markov decision process. For each state-action pair (s, a) , the unknown transition probability vector

$$\theta_{s,a} = (p(s_1 | s, a), \dots, p(s_n | s, a))$$

was assigned a Dirichlet prior. After observing transition counts, the posterior was updated by adding the observed counts to the prior parameters. This gives the agent more than a single transition estimate. It gives a distribution over possible transition models. This distinction is important because uncertainty can guide exploration. Suppose that the agent has observed many transitions from a state-action pair (s, a) . Then the posterior distribution over $\theta_{s,a}$ will usually be concentrated. The agent has a relatively precise belief about what happens after choosing action a in state s . By contrast, if the agent has observed only a few transitions from (s, a) , then the posterior distribution will usually be broad. Many different transition probabilities remain plausible. Thus the posterior represents uncertainty about the environment. This uncertainty is different from the randomness of the environment itself. Even if the transition kernel is

stochastic, the true transition probabilities may be fixed. The Bayesian posterior represents the agent’s uncertainty about those fixed probabilities. This distinction can be summarized as follows. The transition probability

$$p(s' \mid s, a)$$

describes randomness in the environment. The posterior distribution over

$$\theta_{s,a}$$

describes uncertainty in the agent’s knowledge of the environment. This matters for exploration. An agent that only uses a point estimate of the model may behave as if its current estimate is certainly correct. A Bayesian agent can instead take into account that some parts of the environment are still uncertain. It may choose actions not only because they look good under the current estimate, but also because they may provide useful information. There are two broad reasons for choosing an action. The first is exploitation. An action is chosen because, according to the current beliefs, it is likely to produce high return. The second is exploration. An action is chosen because it may reduce uncertainty and improve future decisions. Bayesian reinforcement learning gives a mathematical way to balance these two motives. One simple way to use the posterior is to act according to the posterior mean model. For each state-action pair, define

$$\bar{p}(s_i \mid s, a) = \mathbb{E}[p(s_i \mid s, a) \mid \mathcal{D}_t].$$

Using the Dirichlet posterior from the previous subsection, this is

$$\bar{p}(s_i \mid s, a) = \frac{\alpha_i + N_i(s, a)}{\sum_{j=1}^n \alpha_j + \sum_{j=1}^n N_j(s, a)}.$$

The agent can then treat $\bar{p}$ as an estimated transition model and apply planning methods such as value iteration. This posterior-mean approach is simple, but it does not fully use uncertainty. It collapses the posterior distribution to a single model. If two actions have similar posterior means, but one action is much more uncertain, the posterior mean alone may not capture the value of exploring the uncertain action. A more explicitly Bayesian approach is to sample a model from the posterior. At time t , the agent samples

$$M_t \sim P(M \mid \mathcal{D}_t).$$

It then computes a good policy for the sampled model M_t , and uses that policy to act in the real environment. After observing more transitions, it updates the posterior and repeats the process. This idea is often called posterior sampling or Thompson sampling. In reinforcement learning, it can be viewed as sampling one plausible environment from the posterior and acting optimally for that environment. If an uncertain action could be very good in some plausible models, then it may be selected because one of those models is sampled. As more data are collected, implausible models receive lower posterior probability, and the agent’s behavior becomes more focused. The Bayesian view therefore gives a principled connection between uncertainty and exploration. Actions are not explored merely because of externally added random noise. They

are explored because the posterior says that the agent is still uncertain about their consequences. This is one of the main motivations for Bayesian reinforcement learning. In practice, exact Bayesian reinforcement learning can be computationally difficult. Maintaining a full posterior over all possible Markov decision processes and planning under that posterior may be expensive. However, the conceptual idea is important. Learning is not only about finding one good estimate. It is also about representing what the agent does not yet know, and using that uncertainty to guide decisions.

4.4 Posterior sampling and Thompson sampling

The previous subsection explained that a Bayesian agent maintains a posterior distribution over possible environment models. We now describe one simple way to use this posterior for action selection. The method is called posterior sampling, and in bandit problems it is usually called Thompson sampling. Suppose the agent has observed data $\mathcal{D}_t$. Its uncertainty about the environment is represented by the posterior distribution

$$P(M \mid \mathcal{D}_t).$$

Rather than collapsing this posterior to a single estimated model, posterior sampling draws one model from the posterior,

$$M_t \sim P(M \mid \mathcal{D}_t).$$

The sampled model M_t is treated as if it were the true environment model. The agent then computes a good policy for M_t , for example by applying value iteration if M_t is a finite Markov decision process. The resulting policy is then used to act in the real environment. This produces the following general procedure. Start with a prior distribution over models. At time t , sample a model M_t from the current posterior. Compute a policy π_t that is good for M_t . Use π_t to choose actions and collect new data. Update the posterior using the newly observed transitions. Then repeat. In mathematical terms, the loop is

$$P(M \mid \mathcal{D}_t) \longrightarrow M_t \longrightarrow \pi_t \longrightarrow \mathcal{D}_{t+1} \longrightarrow P(M \mid \mathcal{D}_{t+1}).$$

The posterior is updated as the data set grows, and the sampled models become increasingly concentrated around models that explain the observed experience well. The reason this method explores is that uncertain models are still sampled with positive probability. Suppose an action has not been tried many times. Then the posterior over its consequences may still be broad. Some sampled models may make that action look very good, and in those episodes the agent will try it. If the action turns out to be poor, the posterior will shift away from models where it looked good. If the action turns out to be good, the agent has discovered useful information. This is different from ε -greedy exploration. In ε -greedy exploration, random actions are chosen with some fixed probability, regardless of the structure of the uncertainty. In posterior sampling, exploration is driven by the posterior distribution. The agent explores actions that are uncertain and potentially valuable according to its current beliefs. A simple version can be written for a finite Markov decision process with Dirichlet posteriors over transition probabilities. For each

admissible state-action pair (s, a) , suppose

$$\theta_{s,a} \mid \mathcal{D}_t \sim \text{Dir}(\alpha_1 + N_1(s, a), \dots, \alpha_n + N_n(s, a)).$$

Posterior sampling draws one transition vector for each (s, a) ,

$$\tilde{\theta}_{s,a} \sim \text{Dir}(\alpha_1 + N_1(s, a), \dots, \alpha_n + N_n(s, a)).$$

The collection of sampled transition vectors defines a sampled transition model $\tilde{p}$. The agent can then solve the sampled model using dynamic programming and act according to the resulting policy. This gives a concrete Bayesian analogue of model-based reinforcement learning. Instead of estimating one transition model and planning with it, the agent samples a plausible transition model from its posterior and plans with that sampled model. Repeating this process naturally trades off exploration and exploitation. The same idea can also be applied outside finite transition models. In more complex settings, the posterior may be over reward models, neural network parameters, latent dynamics, or policies. Exact posterior sampling may then be difficult, but the principle remains the same. The agent represents uncertainty through a distribution, samples a plausible hypothesis, acts according to that hypothesis, and updates its belief using new data.

4.5 Example of Thompson sampling for a Bernoulli bandit

We now illustrate Bayesian exploration using the Bernoulli-reward multi-armed bandit example from Xiao. This example is simpler than a full Markov decision process, but it contains the main Bayesian ideas. There is uncertainty about the reward probabilities, the agent maintains a posterior distribution, and actions are chosen by sampling from this posterior. A multi-armed bandit consists of a finite set of actions,

$$\mathcal{A} = \{1, \dots, m\}.$$

At each time step, the agent chooses one action $A_t \in \mathcal{A}$ and receives a reward R_{t+1} . In the Bernoulli-reward setting, each action a has an unknown success probability

$$\theta_a \in [0, 1].$$

If action a is chosen, the reward is sampled according to

$$R_{t+1} \sim \text{Bernoulli}(\theta_a).$$

Thus the reward is either 1 or 0. The value 1 represents success, and the value 0 represents failure. The agent does not know the true parameter θ_a . Instead, it represents uncertainty about θ_a using a probability distribution. A standard prior for a Bernoulli probability is the Beta distribution,

$$\theta_a \sim \text{Beta}(\alpha_a, \beta_a).$$

The parameter α_a can be interpreted as a prior success count, while β_a can be interpreted as a prior failure count. Xiao initializes these parameters by setting

$$\alpha_a = 1, \quad \beta_a = 1$$

for every action a . This corresponds to a uniform prior on $[0, 1]$. Suppose the agent chooses action a and observes reward R_{t+1} . Since R_{t+1} is either 0 or 1, the posterior update is especially simple, being

$$\alpha_a \leftarrow \alpha_a + R_{t+1},$$

and

$$\beta_a \leftarrow \beta_a + 1 - R_{t+1}.$$

Thus observing a reward of 1 increases the success parameter, while observing a reward of 0 increases the failure parameter. Actions that were not chosen are not updated at that time step. After n observations of action a , let S_a be the number of observed successes and let F_a be the number of observed failures. Starting from the prior $\text{Beta}(1, 1)$, the posterior is

$$\theta_a \mid \mathcal{D}_n \sim \text{Beta}(1 + S_a, 1 + F_a).$$

The posterior mean is

$$\mathbb{E}[\theta_a \mid \mathcal{D}_n] = \frac{1 + S_a}{2 + S_a + F_a}.$$

This gives a point estimate of the success probability, but Thompson sampling uses the full posterior distribution rather than only its mean. At each time step, Thompson sampling draws one sampled success probability for each action,

$$\tilde{\theta}_a \sim \text{Beta}(\alpha_a, \beta_a).$$

The agent then chooses an action with the largest sampled value:

$$A_t \in \arg \max_{a \in \mathcal{A}} \tilde{\theta}_a.$$

After the reward is observed, the posterior parameters for the chosen action are updated. The following code follows the structure of Xiao's Thompson sampling solver for the Bernoulli-reward bandit.

```
class ThompsonSamplingSolver:
    def __init__(self, bandit):
        self.bandit = bandit
        self.alphas = np.ones(self.bandit.action_num)
        self.betas = np.ones(self.bandit.action_num)

    def run_one_step(self):
        samples = np.random.beta(self.alphas, self.betas)
        action = np.argmax(samples)
```

```

reward = self.bandit.step(action)

self.alphas[action] += reward
self.betas[action] += 1 - reward

return action, reward

```

The array `alphas` stores the success parameters α_a , and the array `betas` stores the failure parameters β_a . The line

```
samples = np.random.beta(self.alphas, self.betas)
```

draws one sampled success probability from each action’s posterior distribution. The action with the largest sampled value is then selected. After the reward is observed, the posterior parameters of the chosen action are updated. This example shows why Thompson sampling naturally balances exploration and exploitation. If an action has been tried many times, its posterior is usually concentrated, so its sampled values do not vary much. If an action has been tried only a few times, its posterior is broader, so it may occasionally produce a large sampled value and be selected. In this way, uncertainty itself drives exploration. The bandit setting can be viewed as a simplified reinforcement learning problem. There is no state evolution, no long trajectory, and no delayed reward. The agent only chooses actions and observes immediate rewards. Nevertheless, the example captures the Bayesian mechanism clearly. The agent begins with a prior, observes data, updates a posterior, samples from the posterior, and uses those samples to choose actions. In a full Markov decision process, the same principle can be applied to transition models, reward models, or value functions. The mathematical objects become more complicated, but the Bayesian pattern remains the same. The agent represents uncertainty with a prior, updates this uncertainty using observed experience, and uses the resulting posterior to guide decision-making.

4.6 Exploration as uncertainty reduction

The Bernoulli bandit example shows how Bayesian uncertainty can influence action selection. Actions with broad posterior distributions may still be selected because samples from those posteriors can be large. As observations accumulate, the posterior changes, and the agent’s exploration becomes more focused. This gives a different interpretation of exploration from the one used in many standard model-free methods. In standard model-free reinforcement learning, exploration is often introduced by adding randomness to the behavior policy. For example, an ε -greedy policy chooses a random action with probability ε . This ensures that the agent continues to try different actions, but the randomness is not directly tied to what the agent is uncertain about. The Bayesian perspective gives a different interpretation of exploration. The agent maintains a posterior distribution over unknown quantities, such as transition probabilities, reward probabilities, or model parameters. Exploration is then connected to reducing uncertainty in this posterior. Suppose the agent is uncertain about what happens after choosing action a in state s . In the finite transition-model setting, this uncertainty is represented by a posterior

distribution over

$$\theta_{s,a} = (p(s_1 | s, a), \dots, p(s_n | s, a)).$$

If the posterior is broad, then many different transition probabilities are still plausible. Observing more transitions from (s, a) will update the posterior and reduce this uncertainty. This does not mean that the agent should always choose the most uncertain action. An action may be uncertain but irrelevant to obtaining high return. The important question is whether reducing uncertainty can improve future decisions. Thus Bayesian exploration is not only about uncertainty, but about useful uncertainty. A useful way to phrase the issue is that each action has two possible roles. It may produce reward now, and it may produce information that improves future decisions. The first role is exploitation. The second role is exploration. In Bayesian reinforcement learning, both roles are evaluated through the agent's posterior belief. Posterior sampling implements this idea implicitly. If an uncertain action could be valuable under some plausible model, then that action may be selected when such a model is sampled from the posterior. If later observations show that the action is not valuable, the posterior changes and the action is sampled less often. If observations show that the action is valuable, the agent has discovered useful structure in the environment. This is different from purely random exploration. Random exploration may try actions without considering whether they are informative. Bayesian exploration is directed by uncertainty about the environment and by the possible consequences of resolving that uncertainty. The distinction is especially important in sparse-reward problems. If useful rewards are rare, then a greedy agent may never discover them. A Bayesian agent can represent the fact that some poorly explored parts of the environment may still contain high-value outcomes. This uncertainty can justify exploration even when the current point estimate of the value function is low. In practice, exact Bayesian exploration can be computationally difficult. To act optimally under posterior uncertainty, the agent would need to reason about how current actions affect future information and future beliefs. This turns the learning problem itself into a larger decision problem. Posterior sampling is attractive because it avoids explicitly solving this full information-planning problem, while still using posterior uncertainty to guide behavior. The main lesson is that Bayesian reinforcement learning treats uncertainty as part of the state of knowledge of the agent. Learning is not only the process of improving a value estimate or a policy. It is also the process of changing a posterior distribution. Exploration is then the process of collecting data that can make future decisions better.

4.7 From bandits back to Markov decision processes

The Bernoulli bandit example is useful because it isolates the Bayesian mechanism. The agent has uncertainty about unknown reward probabilities, represents this uncertainty with a prior, updates the prior to a posterior after observing data, and uses the posterior to choose actions. However, a bandit is simpler than a full reinforcement learning problem. In a bandit problem, there is no evolving state. At each time step, the agent chooses an action and observes an immediate reward. The action does not move the agent into a new state, and there is no long-term trajectory of states. Thus the main question is which action has the best reward distribution. A Markov decision process is more complicated. The agent observes a state S_t ,

chooses an action A_t , receives a reward R_{t+1} , and moves to a new state S_{t+1} . The action affects not only the immediate reward, but also the future states from which later rewards will be obtained. Therefore the value of an action depends on its long-term consequences. In the Bayesian setting, this means that uncertainty can appear in several places. The agent may be uncertain about the transition probabilities

$$p(s' \mid s, a),$$

the reward distribution

$$p(r \mid s, a),$$

or the full transition-reward kernel

$$p(s', r \mid s, a).$$

Equivalently, the agent may be uncertain about which Markov decision process describes the environment. This creates a new difficulty. In the bandit case, trying an action gives direct information about the reward distribution of that action. In a Markov decision process, trying an action gives information about both rewards and state transitions. This information may affect future planning. An action can be useful because it gives high reward now, because it moves the agent to a valuable future state, or because it reveals information that improves future decisions. Thus Bayesian reinforcement learning in a Markov decision process has two coupled tasks. The agent must learn about the environment, and it must use what it has learned to choose actions. These tasks cannot be fully separated, because the actions chosen by the agent determine what data will be observed. The posterior distribution summarizes the agent's uncertainty after observing data. If M denotes the unknown environment model, then the agent maintains a posterior of the form

$$P(M \mid \mathcal{D}_t).$$

Here $\mathcal{D}_t$ is the data collected through interaction up to time t . Each possible model M specifies how states, actions, rewards, and transitions behave. As the agent observes more transitions, the posterior changes. A policy can then be chosen using this posterior. One possibility is to compute a point estimate of the model, such as the posterior mean model, and plan using that estimate. Another possibility is posterior sampling, where the agent samples a plausible model from the posterior and computes a policy for that sampled model. Both approaches extend ideas from the bandit example to the full Markov decision process setting. The main conceptual difference is that planning now matters. In a bandit problem, choosing an action only affects the reward observed at that time step. In a Markov decision process, choosing an action affects the future state distribution, future rewards, and future information. Bayesian reinforcement learning therefore combines Bayesian inference with sequential decision-making.

4.8 Bayesian model-based reinforcement learning

We now describe the general Bayesian model-based reinforcement learning loop. The agent is uncertain about the environment model. This uncertainty is represented by a posterior dis-

tribution, and the posterior is updated as the agent collects data through interaction. Let M denote the unknown environment model. In a finite Markov decision process, M may specify the transition probabilities, reward distributions, or the full transition-reward kernel. The Bayesian agent begins with a prior distribution over possible models,

$$P(M).$$

After observing data

$$\mathcal{D}_t = \{(S_k, A_k, R_{k+1}, S_{k+1})\}_{k=0}^{t-1},$$

the agent updates this prior to a posterior distribution,

$$P(M \mid \mathcal{D}_t).$$

This posterior represents the agent's current uncertainty about the environment. A model-based reinforcement learning method uses a model to plan. In ordinary model-based reinforcement learning, the agent may learn or estimate one model

$$\hat{M}_t,$$

and then compute a policy using this model. In Bayesian model-based reinforcement learning, the agent instead has a posterior distribution over possible models. The question is how to use this posterior for planning and action selection. One simple approach is to plan with the posterior mean model. For example, if the transition probabilities are unknown, the agent can compute posterior mean transition probabilities

$$\bar{p}(s' \mid s, a) = \mathbb{E}[p(s' \mid s, a) \mid \mathcal{D}_t].$$

The agent then treats $\bar{p}$ as an estimated transition model and applies a planning method such as value iteration. This gives a policy based on the average model under the posterior. This approach is simple, but it compresses the posterior to a single model. It does not fully use the uncertainty represented by the posterior. If two actions have similar posterior mean values, but one action is much more uncertain, planning with only the posterior mean may ignore potentially useful exploration. Another approach is posterior sampling. At time t , the agent samples a model from the posterior,

$$M_t \sim P(M \mid \mathcal{D}_t).$$

It then computes a policy that is good for the sampled model M_t . If M_t is a finite Markov decision process, this policy can be computed using dynamic programming. The agent then follows this policy for some amount of interaction, collects new data, and updates the posterior. The Bayesian model-based loop can be written as

$$P(M \mid \mathcal{D}_t) \longrightarrow M_t \longrightarrow \pi_t \longrightarrow \mathcal{D}_{t+1} \longrightarrow P(M \mid \mathcal{D}_{t+1}).$$

The posterior produces a plausible model. The sampled model produces a policy. The policy produces new experience. The new experience updates the posterior. This loop makes the role of uncertainty explicit. If the agent has little data about some state-action pair, then many transition or reward models may still be plausible. Some sampled models may make that state-action pair look valuable, which can cause the agent to try it. If the observed data do not support that optimistic model, the posterior changes. If the observed data do support it, the agent has discovered useful information. Bayesian model-based reinforcement learning therefore combines two operations. The first operation is inference, where the agent updates its posterior belief about the environment model. The second operation is planning, where the agent uses its current belief to choose actions. These two operations interact because the chosen actions determine which data the agent will observe next. This is different from the model-free methods discussed earlier. In model-free methods, the agent often updates value estimates or policy parameters directly from sampled transitions. In Bayesian model-based reinforcement learning, the agent first updates a belief over environment models, and then uses this belief to plan. The learned object is not only a value function or a policy, but also a posterior distribution over possible environments. The advantage of this approach is that uncertainty is represented explicitly. This can make exploration more principled, especially when data are limited. The disadvantage is computational cost. Maintaining a posterior over models can be difficult, and planning under this posterior can be much more expensive than updating a table or a neural network. For this reason, practical Bayesian reinforcement learning often relies on approximations.

4.9 Optimism and upper-confidence planning

Posterior sampling is one way to use uncertainty for exploration. The agent samples a plausible model from its posterior and acts according to that sampled model. Another important idea is optimism under uncertainty. Instead of sampling one plausible model, the agent acts as if uncertain quantities might take favorable values, as long as those values are still compatible with the data. The basic idea is easiest to understand in terms of action values. Suppose the agent has an estimate

$$\hat{Q}(s, a)$$

of the value of choosing action a in state s . If this estimate is based on many observations, then the agent may be fairly confident in it. If it is based on few observations, then the estimate is uncertain. An optimistic agent adds a bonus to uncertain estimates

$$Q_{\text{UCB}}(s, a) = \hat{Q}(s, a) + b(s, a).$$

The term $b(s, a)$ is an exploration bonus. It is large when the agent has little information about (s, a) , and it becomes smaller as more data are collected. The action is then chosen greedily with respect to the optimistic value

$$A_t \in \arg \max_{a \in \mathcal{A}(S_t)} Q_{\text{UCB}}(S_t, a).$$

This rule encourages exploration because poorly understood actions receive a temporary bonus. If such an action turns out to be useful, the agent has discovered something valuable. If it turns out to be poor, the bonus decreases as more observations are collected. This idea is called upper-confidence exploration. The word upper refers to the fact that the agent uses an optimistic upper estimate of the value, rather than only the current mean estimate. In a simple bandit problem, one often writes an upper-confidence action value as

$$\hat{\mu}_a + c\sqrt{\frac{\log t}{N_t(a)}}.$$

Here $\hat{\mu}_a$ is the empirical mean reward of action a , $N_t(a)$ is the number of times action a has been selected before time t , and $c > 0$ controls the strength of exploration. If an action has been selected only a few times, then $N_t(a)$ is small and the bonus is large. If an action has been selected many times, the bonus becomes smaller. The same principle can be extended from bandits to Markov decision processes. In a finite Markov decision process, the agent may estimate transition probabilities and rewards from data. If some state-action pairs have been visited only rarely, then the estimates associated with them are uncertain. Upper-confidence planning incorporates this uncertainty into the planning problem. One way to express the idea is to add a bonus to the reward used during planning. Instead of planning with an estimated reward

$$\hat{r}(s, a),$$

the agent plans with an optimistic reward

$$\hat{r}(s, a) + b(s, a).$$

The value iteration update then becomes

$$V_{k+1}(s) = \max_{a \in \mathcal{A}(s)} \left[\hat{r}(s, a) + b(s, a) + \gamma \sum_{s' \in \mathcal{S}} \hat{p}(s' | s, a) V_k(s') \right].$$

The bonus makes uncertain actions look better during planning. As the agent gathers more data from those actions, the uncertainty decreases, and the bonus shrinks. This is closely related to the Bayesian perspective, but it is not exactly the same as maintaining and sampling from a full posterior. In posterior sampling, uncertainty is represented by a posterior distribution, and actions are chosen by sampling a plausible model. In upper-confidence planning, uncertainty is represented through confidence intervals or exploration bonuses, and actions are chosen by planning optimistically. Both methods use uncertainty to guide exploration. Posterior sampling explores because uncertain models are sometimes sampled. Upper-confidence planning explores because uncertain actions receive optimistic bonuses. In both cases, the agent is not merely adding random noise to its behavior. Exploration is directed toward parts of the environment where the agent has insufficient information and where learning could matter for future return. In a finite Markov decision process, an upper-confidence method often uses visit counts. Let

$$N_t(s, a)$$

be the number of times the state-action pair (s, a) has been observed before time t . A typical exploration bonus has the qualitative form

$$b_t(s, a) \propto \frac{1}{\sqrt{N_t(s, a)}}.$$

The exact formula depends on the algorithm and on the desired confidence guarantees. The important feature is that the bonus is large when $N_t(s, a)$ is small and small when $N_t(s, a)$ is large. This gives a useful interpretation of optimism. The agent treats poorly explored parts of the environment as potentially better than their current estimates suggest. This prevents the agent from prematurely ignoring actions that have not yet been tried enough. Once those actions are explored, the data either confirm that they are useful or reduce the optimism attached to them. Optimism and posterior sampling are two complementary ways to connect uncertainty with exploration. Posterior sampling represents uncertainty through a distribution and samples from it. Upper-confidence planning represents uncertainty through bonuses or confidence bounds and plans optimistically. Both approaches are especially important when data are limited, rewards are sparse, or exploration by random action selection is inefficient.

4.10 Uncertainty in model-free reinforcement learning

Model-free reinforcement learning methods do not learn an explicit model of the environment. Instead, they learn value functions, action-value functions, or policies directly from sampled experience. In the tabular methods discussed earlier, the learned object was often a table such as

$$V(s)$$

or

$$Q(s, a).$$

These tables were updated from observed returns or temporal-difference targets. From a Bayesian perspective, the important point is that these tables are point estimates. A standard Q-learning algorithm stores one number for each state-action pair,

$$Q(s, a).$$

This number represents the current estimate of the long-term value of choosing action a in state s . However, it does not by itself say how uncertain the agent is about this estimate. This is a limitation. Two state-action pairs may have the same estimated value but very different levels of uncertainty. One estimate may be based on thousands of observations, while another may be based on only one or two observations. A standard point estimate does not distinguish these cases unless additional information, such as visit counts, is stored. A Bayesian or uncertainty-aware version of value-based reinforcement learning tries to represent this missing information. Instead of treating $Q(s, a)$ as a single known number, the agent may represent uncertainty about the value of (s, a) . Conceptually, one may write this as a distribution over possible action values,

$$P(Q(s, a) \mid \mathcal{D}_t).$$

This distribution represents the agent’s belief about the value of (s, a) after observing the data $\mathcal{D}_t$. If this distribution is concentrated, then the agent is confident about the value estimate. If it is broad, then many different values are still plausible. This uncertainty can be used for exploration. For example, an uncertain action may be tried because it could still have high value under some plausible interpretation of the data. This gives a Bayesian interpretation of exploration in model-free learning. In ordinary ε -greedy Q-learning, exploration is produced by choosing random actions with probability ε . In an uncertainty-aware method, exploration can instead depend on how uncertain the agent is about each action. Actions with uncertain values may receive bonuses, be sampled optimistically, or be selected according to a posterior distribution. One simple uncertainty-aware modification is to store both an estimate and a measure of uncertainty. For example, one may use

$$\hat{Q}(s, a)$$

as the current value estimate and

$$b(s, a)$$

as an uncertainty bonus. The agent then chooses actions according to

$$\hat{Q}(s, a) + b(s, a).$$

This resembles the upper-confidence idea discussed above. The bonus is large when the value estimate is uncertain and becomes smaller as more experience is collected. Another approach is to maintain several possible value functions. Suppose the agent has an ensemble

$$Q_1, Q_2, \dots, Q_K.$$

Each Q_i is trained from sampled experience, possibly using different initialization, different data subsets, or different random seeds. If the ensemble members agree at a state-action pair, the agent may treat the value estimate as relatively certain. If they disagree, the agent has evidence that the value estimate is uncertain. The spread of the ensemble can then be used as an approximate measure of uncertainty. This is not exact Bayesian inference, but it is Bayesian in spirit. A full Bayesian method would maintain a posterior distribution over value functions. An ensemble gives a finite collection of plausible value functions, which can be viewed as an approximation to such a posterior. This approximation is often much easier to implement than exact Bayesian inference. Uncertainty can also be attached to returns. In Monte Carlo learning, the return following a state-action pair may vary from episode to episode. The sample average estimates the mean return, but the variability of the observed returns contains additional information. A Bayesian treatment would not only estimate the mean return, but also represent uncertainty about that mean. With few observed returns, the posterior over the mean is broad. With many observed returns, it becomes more concentrated. The main lesson is that model-free reinforcement learning can be viewed in two ways. In the standard view, the agent updates point estimates of values or policies from sampled experience. In the Bayesian view, the agent also represents uncertainty about those estimates. This uncertainty can be used to guide

exploration, quantify confidence, and avoid acting as if poorly supported estimates were certain. Thus Bayesian model-free reinforcement learning does not require us to abandon the algorithms from the previous chapters. Instead, it changes what the agent keeps track of. Standard Q-learning asks what value estimate should be assigned to (s, a) . A Bayesian or uncertainty-aware version also asks how certain the agent should be about that estimate.

4.11 Uncertainty in deep reinforcement learning

Deep reinforcement learning replaces tables by neural networks. In deep Q-learning, the action-value function is represented by a network

$$\hat{q}(s, a; w).$$

In actor-critic methods, the policy may be represented by an actor network

$$\pi_{\theta}(a \mid s),$$

and the value function may be represented by a critic network

$$\hat{v}(s; w).$$

These networks are usually trained as point estimates. After training, the agent has one set of parameters w , or one set of policy parameters θ . The network gives predictions, but it does not automatically say how uncertain those predictions are. This matters because neural networks can be confidently wrong. A network may output a large action value for a state-action pair even if the agent has little relevant experience near that input. In supervised learning, this is already a problem. In reinforcement learning, it is even more important because the agent uses its predictions to choose actions, and those actions determine what data will be collected next. A Bayesian treatment would place a probability distribution over the network parameters. For a value network, this would mean representing a posterior of the form

$$P(w \mid \mathcal{D}_t).$$

Instead of one value function $\hat{q}(s, a; w)$, the agent would have a distribution over possible value functions induced by the posterior over w . For a policy network, one could similarly consider a posterior over policy parameters,

$$P(\theta \mid \mathcal{D}_t).$$

This posterior would represent uncertainty about which policy parameters are supported by the observed data. Exact Bayesian inference for neural networks is usually difficult. Neural networks have many parameters, and the posterior distribution over those parameters is typically high-dimensional and analytically intractable. For this reason, Bayesian deep reinforcement learning often uses approximations. One common approximation is to use ensembles. Instead of training

one value network, the agent trains several networks,

$$\hat{q}_1, \hat{q}_2, \dots, \hat{q}_K.$$

Each network may be initialized differently or trained on different samples from experience. The ensemble can then be interpreted as a finite collection of plausible value functions. If the networks agree on the value of a state-action pair, the agent may treat the prediction as relatively certain. If the networks disagree, this indicates uncertainty. The disagreement in an ensemble can be used for exploration. For example, the agent may choose actions according to an optimistic estimate such as

$$\frac{1}{K} \sum_{i=1}^K \hat{q}_i(s, a) + \lambda \text{Std}(\hat{q}_1(s, a), \dots, \hat{q}_K(s, a)),$$

where $\lambda > 0$ controls the strength of the uncertainty bonus. The mean term estimates value, while the standard deviation term estimates uncertainty. Actions with uncertain but potentially high value may therefore be explored. Another possibility is posterior sampling with ensembles. At the beginning of an episode, the agent selects one network from the ensemble and acts greedily with respect to that network. This imitates Thompson sampling. Instead of sampling an exact model or value function from a true posterior, the agent samples one approximate hypothesis from the ensemble. Uncertainty can also be represented in model-based deep reinforcement learning. In this setting, the agent learns a neural-network dynamics model. Such a model may predict the next state and reward from the current state and action,

$$(\hat{S}_{t+1}, \hat{R}_{t+1}) = f_w(S_t, A_t).$$

A Bayesian or uncertainty-aware version does not only predict one next state and reward. It also tries to represent uncertainty about the prediction. This can be done with probabilistic neural networks, ensembles of dynamics models, or distributions over latent variables. This distinction is important. Some uncertainty comes from the environment itself. Even with perfect knowledge, the next state may be random. This is sometimes called aleatoric uncertainty. Other uncertainty comes from lack of knowledge. The agent may simply not have enough data to know what will happen. This is often called epistemic uncertainty. Bayesian methods are especially concerned with epistemic uncertainty, because it can be reduced by collecting more data. In deep reinforcement learning, epistemic uncertainty is closely connected to exploration. If the agent encounters states far from its previous experience, its predictions should be uncertain. A point-estimate neural network may not reveal this. An uncertainty-aware method can detect that the state is unfamiliar and encourage exploration or caution. Uncertainty is also relevant for safety. If the agent is unsure about the consequences of an action, it may be inappropriate to act greedily with respect to a single value estimate. In some settings, one may prefer conservative behavior, where uncertain actions are avoided unless there is enough evidence that they are safe. In other settings, one may prefer exploratory behavior, where uncertain actions are tried because they may lead to high reward. The Bayesian framework makes this distinction explicit by representing uncertainty rather than hiding it inside a point estimate. The main lesson is that

deep reinforcement learning usually learns functions, while Bayesian deep reinforcement learning tries to learn distributions over functions or approximate uncertainty about those functions. Exact Bayesian neural-network reinforcement learning is often computationally difficult, but approximations such as ensembles, posterior sampling, uncertainty bonuses, and probabilistic dynamics models can capture part of the same idea. They allow the agent to reason not only about what it predicts, but also about how much it should trust those predictions.

4.12 The Bayes-adaptive viewpoint

The previous sections described Bayesian reinforcement learning as a process of maintaining and updating uncertainty about the environment. We now describe a more unified viewpoint. The key idea is that the agent’s belief is itself part of the information needed for decision-making. In an ordinary Markov decision process, the agent observes a state S_t , chooses an action A_t , receives a reward R_{t+1} , and moves to a next state S_{t+1} . If the environment model is known, then the current state contains the information needed to predict the distribution of the next state and reward. In Bayesian reinforcement learning, the environment model is not known. The agent must also keep track of its current belief about the model. Let M denote the unknown environment model. After observing data $\mathcal{D}_t$, the agent’s belief is represented by the posterior distribution

$$P(M \mid \mathcal{D}_t).$$

Thus, at time t , the agent’s information consists of both the current environment state S_t and the current posterior belief. We may write this informally as

$$(S_t, P(M \mid \mathcal{D}_t)).$$

This pair is sometimes called a belief state. It describes both where the agent is in the environment and what the agent currently believes about the environment. When the agent takes an action, two things happen. First, the environment state changes. The agent observes a transition

$$(S_t, A_t, R_{t+1}, S_{t+1}).$$

Second, the agent’s belief changes. The new transition is added to the data set, producing

$$\mathcal{D}_{t+1} = \mathcal{D}_t \cup \{(S_t, A_t, R_{t+1}, S_{t+1})\}.$$

The posterior is then updated from

$$P(M \mid \mathcal{D}_t)$$

to

$$P(M \mid \mathcal{D}_{t+1}).$$

Thus an action affects not only the next physical state, but also the agent’s future knowledge. This is the central idea of the Bayes-adaptive viewpoint. The agent is not only moving through the environment. It is also moving through a space of beliefs. The state relevant for optimal decision-making is therefore not just S_t , but the combination of the environment state and the

posterior belief. In principle, one can define a larger decision process whose states are belief states. In this larger process, choosing an action produces a reward, a new environment state, and an updated posterior. The transition rule of this larger process is determined by the current posterior, the possible models, and Bayes' rule. This construction is called the Bayes-adaptive Markov decision process. The Bayes-adaptive formulation gives a principled solution concept. An optimal Bayesian agent should choose actions that maximize expected return while taking into account how actions affect future information. This means that exploration and exploitation are not separate heuristics. They are both parts of the same planning problem. For example, an action may have low immediate expected reward but may reveal important information about the environment. If this information can improve future decisions, then the action may be valuable from the Bayes-adaptive perspective. The value of an action therefore includes both its direct reward effect and its informational effect. This explains why exact Bayesian reinforcement learning is difficult. To act optimally, the agent would need to plan over future states, rewards, observations, and posterior updates. The posterior can be high-dimensional, and the space of possible future beliefs can be very large. Even when the original Markov decision process is finite, the corresponding belief-state process may be computationally expensive. Posterior sampling and upper-confidence methods can be viewed as approximations to this ideal. Posterior sampling avoids planning explicitly over future beliefs by sampling a plausible model and acting as if it were true. Upper-confidence methods avoid maintaining a full belief-state planner by adding bonuses to uncertain actions. Both methods use uncertainty, but neither usually solves the full Bayes-adaptive planning problem exactly. The Bayes-adaptive viewpoint is nevertheless important conceptually. It shows what Bayesian reinforcement learning is trying to approximate. The agent should not only ask which action looks best under the current estimate. It should also ask how actions change what it will know later. In this sense, Bayesian reinforcement learning treats learning itself as part of the sequential decision problem.

4.13 Implementation example comparing exploration strategies

We finish the chapter with a small implementation example based on the Bernoulli-reward multi-armed bandit case study in Xiao's book. The purpose is to compare four different ways of choosing actions under uncertainty. The first method is ϵ -greedy exploration, which adds random exploration to a greedy policy. The second method is UCB1, which chooses actions using an upper-confidence bonus. The third method is Bayesian UCB, which uses posterior quantiles from a Beta distribution. The fourth method is Thompson sampling, which samples from the posterior distribution and acts greedily with respect to the sampled values. The environment is a Bernoulli bandit with m actions. Each action a has an unknown success probability θ_a . If action a is chosen, the reward is sampled according to

$$R_{t+1} \sim \text{Bernoulli}(\theta_a).$$

The agent does not know the vector

$$(\theta_1, \dots, \theta_m).$$

It only observes rewards from the actions it chooses. This makes the example simple, but it contains the essential exploration problem. The agent must decide whether to exploit actions that currently look good or explore actions that are still uncertain. We compare the algorithms using pseudo-regret. If

$$\theta_* = \max_{a \in \mathcal{A}} \theta_a$$

is the best success probability, and action A_t is chosen at time t , then the one-step pseudo-regret is

$$\theta_* - \theta_{A_t}.$$

The cumulative pseudo-regret after T steps is

$$\sum_{t=0}^{T-1} (\theta_* - \theta_{A_t}).$$

Low regret means that the agent quickly learns to choose actions whose reward probabilities are close to optimal. The following code implements the environment and the four exploration strategies.

```
import numpy as np
from scipy.stats import beta
import matplotlib.pyplot as plt

class BernoulliBandit:
    def __init__(self, probs, seed=None):
        self.probs = np.array(probs)
        self.action_num = len(probs)
        self.best_prob = np.max(self.probs)
        self.rng = np.random.default_rng(seed)

    def step(self, action):
        return int(self.rng.random() < self.probs[action])

class Solver:
    def __init__(self, bandit):
        self.bandit = bandit
        self.counts = np.zeros(self.bandit.action_num)
        self.estimates = np.zeros(self.bandit.action_num)
        self.regrets = []
        self.total_regret = 0.0

    def update_regret(self, action):
        self.total_regret += self.bandit.best_prob - self.bandit.probs[action]
```

```

        self.regrets.append(self.total_regret)

def update_estimate(self, action, reward):
    self.counts[action] += 1
    n = self.counts[action]
    self.estimated[action] += (reward - self.estimated[action]) / n

def run(self, steps):
    for _ in range(steps):
        action = self.run_one_step()
        self.update_regret(action)

class EpsilonGreedySolver(Solver):
    def __init__(self, bandit, epsilon=0.01):
        super().__init__(bandit)
        self.epsilon = epsilon

    def run_one_step(self):
        if np.random.random() < self.epsilon:
            action = np.random.randint(self.bandit.action_num)
        else:
            action = int(np.argmax(self.estimated))

        reward = self.bandit.step(action)
        self.update_estimate(action, reward)

        return action

class UCB1Solver(Solver):
    def __init__(self, bandit, coefficient=1.0):
        super().__init__(bandit)
        self.coefficient = coefficient
        self.total_count = 0

    def run_one_step(self):
        self.total_count += 1

        for action in range(self.bandit.action_num):
            if self.counts[action] == 0:
                reward = self.bandit.step(action)

```

```

        self.update_estimate(action, reward)
        return action

    bonus = self.coefficient * np.sqrt(
        np.log(self.total_count) / self.counts
    )
    action = int(np.argmax(self.estimateds + bonus))

    reward = self.bandit.step(action)
    self.update_estimate(action, reward)

    return action

class BayesianUCBSolver(Solver):
    def __init__(self, bandit, quantile=0.95):
        super().__init__(bandit)
        self.quantile = quantile
        self.alphas = np.ones(self.bandit.action_num)
        self.betas = np.ones(self.bandit.action_num)

    def run_one_step(self):
        upper_bounds = beta.ppf(self.quantile, self.alphas, self.betas)
        action = int(np.argmax(upper_bounds))

        reward = self.bandit.step(action)

        self.alphas[action] += reward
        self.betas[action] += 1 - reward
        self.update_estimate(action, reward)

        return action

class ThompsonSamplingSolver(Solver):
    def __init__(self, bandit):
        super().__init__(bandit)
        self.alphas = np.ones(self.bandit.action_num)
        self.betas = np.ones(self.bandit.action_num)

    def run_one_step(self):
        samples = np.random.beta(self.alphas, self.betas)

```

```

    action = int(np.argmax(samples))

    reward = self.bandit.step(action)

    self.alphas[action] += reward
    self.betas[action] += 1 - reward
    self.update_estimate(action, reward)

    return action

```

The ε -greedy solver uses random actions with probability ε . It does not represent uncertainty explicitly. It explores because randomness is forced into the behavior policy. UCB1 explores differently. It adds a bonus to actions that have been tried few times. Bayesian UCB uses the posterior distribution more directly. For each action, it computes an upper posterior quantile and chooses the action with the largest quantile. Thompson sampling samples one possible success probability from each posterior and chooses the action with the largest sampled value. The following code runs the four solvers on the same bandit instance and plots cumulative pseudo-regret.

```

probs = [0.15, 0.30, 0.45, 0.55, 0.60]
steps = 5000

solvers = {
    "epsilon-greedy": EpsilonGreedySolver(
        BernoulliBandit(probs, seed=1),
        epsilon=0.01
    ),
    "UCB1": UCB1Solver(
        BernoulliBandit(probs, seed=1),
        coefficient=1.0
    ),
    "Bayesian UCB": BayesianUCBSolver(
        BernoulliBandit(probs, seed=1),
        quantile=0.95
    ),
    "Thompson sampling": ThompsonSamplingSolver(
        BernoulliBandit(probs, seed=1)
    ),
}

for solver in solvers.values():
    solver.run(steps)

plt.figure(figsize=(8, 5))

```

```

for name, solver in solvers.items():
    plt.plot(solver.regrets, label=name)

plt.xlabel("time step")
plt.ylabel("cumulative pseudo-regret")
plt.legend()
plt.tight_layout()
plt.show()

```

The expected qualitative behavior is as follows. The ε -greedy method eventually learns which actions are good, but it keeps exploring randomly. This means that it continues to make suboptimal choices even after it has enough information. UCB1 usually explores more systematically, because the exploration bonus decreases as the visit count increases. Bayesian UCB behaves similarly, but its optimism comes from posterior quantiles rather than from an explicit count-based formula. Thompson sampling often performs well because it explores actions in proportion to posterior plausibility. If an action is uncertain and could be optimal, it may be sampled and selected. If later observations make it unlikely to be optimal, it is selected less often. This example summarizes the main ideas of the chapter in code. The ε -greedy method represents the standard non-Bayesian approach of adding random exploration. UCB1 represents optimism under uncertainty. Bayesian UCB represents posterior uncertainty through upper posterior quantiles. Thompson sampling represents posterior uncertainty through sampling. All four algorithms face the same problem, but they use different information when deciding which action to try next. The main limitation of the example is that it is a bandit problem rather than a full Markov decision process. There are no states, transition probabilities, or delayed rewards. Nevertheless, the comparison is useful because the exploration mechanisms are clearly visible. In a full Markov decision process, the same principles can be applied to transition models, reward models, value functions, or neural-network predictions. The mathematical objects become larger, but the central idea remains the same. An agent should not only estimate what is best. It should also reason about how uncertain those estimates are.

4.14 Summary of the Bayesian perspective

This chapter added a Bayesian layer to the reinforcement learning framework developed earlier. In the previous chapters, the environment was described probabilistically through a transition-reward kernel

$$p(s', r \mid s, a),$$

and policies generated actions according to probabilities

$$\pi(a \mid s).$$

Once the environment, policy, and initial distribution were fixed, they determined a probability law over trajectories. The Bayesian perspective changes what is treated as uncertain. Instead of assuming that the environment model is known, or learning only a single point estimate, the

agent represents uncertainty about unknown quantities using probability distributions. These unknown quantities may be transition probabilities, reward probabilities, full environment models, value functions, policies, or neural-network parameters. The basic Bayesian mechanism is the update from prior to posterior. In a discrete model class

$$\mathfrak{M} = \{M_1, M_2, M_3, \dots\},$$

the prior probabilities

$$\mathbb{P}(M_i)$$

describe how plausible each model is before observing data. After observing experience

$$\mathcal{D}_t,$$

Bayes' rule gives the posterior

$$\mathbb{P}(M_i \mid \mathcal{D}_t) = \frac{\mathbb{P}(\mathcal{D}_t \mid M_i)\mathbb{P}(M_i)}{\mathbb{P}(\mathcal{D}_t)}.$$

The posterior describes how plausible each model is after the observed data have been taken into account. In finite transition models, this update can be made explicit. For each state-action pair (s, a) , the unknown transition probabilities can be collected into a vector

$$\theta_{s,a} = (p(s_1 \mid s, a), \dots, p(s_n \mid s, a)).$$

A Dirichlet prior over this vector is updated by adding observed transition counts. Thus Bayesian learning can be interpreted as updating pseudo-counts into posterior counts. The posterior is useful because it represents uncertainty, not only a single estimate. If the posterior is concentrated, the agent has strong evidence about the unknown quantity. If the posterior is broad, many possibilities remain plausible. This distinction is important for exploration. Posterior sampling uses the posterior directly. The agent samples a plausible model or parameter value from its current posterior, acts according to that sample, observes new data, and updates the posterior. In the Bernoulli bandit example, this became Thompson sampling. Each action had an unknown success probability

$$\theta_a,$$

a Beta posterior was maintained for each action, and actions were chosen by sampling from these posteriors. Optimism and upper-confidence planning use uncertainty in a different way. Instead of sampling a model, the agent adds exploration bonuses to uncertain actions. A typical optimistic action value has the form

$$\hat{Q}(s, a) + b(s, a),$$

where $b(s, a)$ is large when the agent has little information about (s, a) . This encourages the agent to explore actions whose current value estimates are uncertain. The Bayesian perspective also applies to model-free reinforcement learning. Standard methods such as Monte Carlo

learning, SARSA, and Q-learning learn point estimates such as

$$V(s)$$

or

$$Q(s, a).$$

A Bayesian or uncertainty-aware version asks not only what the estimated value is, but also how uncertain that estimate should be. This uncertainty can be represented by posterior distributions, confidence bonuses, visit counts, or ensembles of value functions. In deep reinforcement learning, the same issue appears for neural networks. A standard deep RL method often learns one value network

$$\hat{q}(s, a; w),$$

or one policy network

$$\pi_{\theta}(a \mid s).$$

A Bayesian deep RL method tries to represent uncertainty over parameters, predictions, policies, or learned dynamics models. Exact Bayesian inference for neural networks is usually difficult, so practical methods often use approximations such as ensembles, uncertainty bonuses, posterior sampling approximations, or probabilistic dynamics models. The Bayes-adaptive viewpoint gives the most unified interpretation. In standard reinforcement learning, the state S_t describes where the agent is in the environment. In Bayesian reinforcement learning, the agent's belief about the environment is also relevant. The effective state of knowledge is therefore described by something like

$$(S_t, \mathbb{P}(M \mid \mathcal{D}_t)).$$

When the agent acts, it changes both the environment state and its future belief. Thus learning itself becomes part of the sequential decision problem. This is the conceptual core of Bayesian reinforcement learning. The agent is not only trying to find a good value function or a good policy. It is also trying to reason about what it does not know. The posterior distribution represents uncertainty, and exploration is the process of collecting data that can make future decisions better. The main advantage of the Bayesian perspective is that it gives a principled way to connect learning, uncertainty, and exploration. The main difficulty is computational. Maintaining exact posterior distributions and planning under posterior uncertainty can be expensive or intractable. For this reason, practical Bayesian reinforcement learning often uses approximations. The earlier chapters introduced the standard reinforcement learning objects. This chapter showed how those objects change when uncertainty is represented explicitly. The transition model may become a posterior over models. A value estimate may become a distribution over possible values. A neural network may become an approximate distribution over functions. The result is a richer view of reinforcement learning, where the agent learns not only what to do, but also how much it should trust what it currently believes.

5 BONUS EXAMPLE: AGENT PLAYS POKÉMON

As a final example of how reinforcement learning can be used, we turn to (the author's favorite franchise) Pokémon. This example is based on the excellent Youtube video made by Peter Whidden, called "Training AI to Play Pokemon with Reinforcement Learning". We highly recommend checking this video out, as it goes into further detail and also includes a guide on how to run the model yourself. All his code can be found in his public Github repository, which is linked in the description of the video. We encourage students to clone his repo and try to improve on his code! In this text, we will not go into overly technical details on the implementation but rather look at the heuristic process of training the agent. As mentioned, more details can be found in the original youtube video and the corresponding Github repository.

This example regards training an agent to play Pokémon, specifically Pokémon Red, released for the Nintendo Gameboy in 1996. This is a substantially more complicated problem than our previous examples, as the number of possible actions in each state are extremely vast. Having said that, it also serves as an excellent test, since the game has well defined possible actions, as well as a clear end goal. Another strong point of this example is that it illustrates many of the difficulties in reinforcement learning as well as contrasts between different reinforcement learning settings we have covered so far. This example will likely be most interesting for those who have played similar games before, nevertheless, we now give a brief introduction to the mechanics of the game in hopes that those who have not played the game can still follow the example. The point of the game is traveling from town to town in a virtual world, while catching and training a team of Pokémon used for battling other Pokémon trainers. In each town there is a powerful trainer called a gym leader, which needs to be beat. After beating eight gym leaders, the player can take on the Pokémon league, which serves as the final challenge of the game. Hence, the ultimate task is beating the Pokémon league. To do this, the player must navigate a two-dimensional grid, interact with characters, use menus, catch and train Pokémon, win battles, recover after losing, and make progress through a long sequence of events. This makes the environment considerably more complicated than the FrozenLake example, which has a similar grid structure. Nevertheless, the basic reinforcement learning structure is the same. At time t , the agent receives an observation O_t , chooses an action A_t , receives a reward R_{t+1} , and obtains a new observation O_{t+1} . We may summarize the setup as

$$O_t \longrightarrow A_t \longrightarrow (R_{t+1}, O_{t+1}).$$

The observation is based mainly on the image displayed by the Game Boy (or in this case an emulator on your computer), together with a small amount of additional information about recent progress. The available actions correspond to Game Boy controls such as moving up, down, left, or right and pressing the A or B buttons.

The grid structure gives an immediate connection to FrozenLake. In both environments, the agent frequently chooses a direction and moves between locations on a grid. In FrozenLake, however, each grid position could be represented by a small discrete state. Pokémon Red con-

tains many connected maps, animated objects, characters, menus, battles, and hidden game variables. Two visually similar observations can also correspond to very different situations. A screen shown during a battle does not provide the full history of how the agent reached that battle, which Pokémon are useful, or which objectives have already been completed.

For this reason, it is more accurate to write O_t for the agent’s observation than S_t for the complete state of the game. The emulator has an underlying internal state, but the agent only observes a limited representation of it. The problem is therefore closer to a partially observable Markov decision process than to the fully observable FrozenLake example.

The policy used for this model is represented by a convolutional neural network. Heuristically, the network implements a stochastic policy

$$\pi_{\theta}(a \mid o),$$

which assigns probabilities to the possible button presses given the current observation. A value network also estimates

$$\hat{v}(o; w),$$

which predicts the expected future return from the observation. PPO uses sampled trajectories to update both the policy and the value estimate.

The network is not recurrent and therefore does not maintain a learned internal memory over the entire game. Instead, the three most recent screens are stacked together. This gives the network a small amount of temporal information. Additional progress information, including quantities related to health, levels, and exploration, is also encoded into the observation. This illustrates an important practical point. The theoretical state of an MDP is often replaced in deep reinforcement learning by an engineered observation that contains enough information for useful behavior, even when it does not contain the complete state.

5.0.1 Is the agent model-based or model-free?

The project uses a Game Boy emulator, which is a program capable of simulating the game. It may therefore be tempting to call the method model-based. This would confuse the simulation environment with the model used by the learning algorithm.

PPO is a model-free reinforcement learning algorithm. The agent is not given the transition probabilities

$$p(s', r \mid s, a),$$

and it does not learn a separate predictive model

$$\hat{p}(s', r \mid s, a)$$

that can be used to simulate possible futures. The agent cannot ask the emulator what would happen under several alternative button presses and then plan over those hypothetical outcomes. It must choose an action, observe what actually happens, and learn from the resulting trajectory.

The emulator generates experience, but it is not used by the policy as an explicit world

model. This gives a useful distinction

$$\text{simulated environment} \neq \text{model-based agent}.$$

An algorithm may be trained inside a perfectly implemented simulator and still be model-free. The relevant question is whether the learning algorithm uses a model of the transition dynamics for prediction and planning.

This differs from the value-iteration example on FrozenLake. There, the algorithm had access to the environment model through the transition probabilities stored in `env.P`. It could evaluate the possible next states and rewards associated with each action. Value iteration was therefore model-based.

The Pokémon agent instead learns directly from sampled interaction. It collects trajectories from the emulator, estimates which actions produced higher returns, and changes its policy accordingly. A model-based alternative might learn a world model from the observed screens and button presses, use that model to predict future observations, and plan inside the learned model. Methods such as Dreamer and MuZero move in this direction, however, we will not cover those in this text.

Model-free does not mean that the policy learns nothing about the structure of the environment. Regularities in the game can become encoded implicitly in the policy parameters. In the video, the agent eventually discovers a repeatable sequence of actions that manipulates the game’s deterministic random-number generation and allows it to catch a Pokémon reliably. The agent does not possess an explicit equation for the emulator dynamics. Nevertheless, its policy has learned an action pattern that exploits those dynamics.

5.0.2 Exploration and reward shaping

Pokémon Red has an extremely sparse natural objective. The ultimate goal of completing the game may require many thousands of decisions. An agent pressing nearly random buttons is very unlikely to complete a gym battle or reach a distant city by accident. Giving reward only for completing the game would therefore provide almost no useful learning signal.

The project addresses this problem using reward shaping. Instead of rewarding only the final objective, the agent receives intermediate rewards for behaviors that are expected to support progress. Two especially important rewards are exploration and increases in the combined levels of the agent’s Pokémon. The exploration reward is based on novelty. The system keeps track of screens that have already been observed. When the current screen is sufficiently different from the stored screens, the agent receives a reward for discovering something new. This encourages the agent to leave familiar areas and explore the map. The idea is similar to rewarding a FrozenLake agent for visiting previously unseen grid cells. In FrozenLake, however, determining whether a state is new is easy because the state is simply a grid index. In Pokémon, novelty must be estimated from images. Animated water, moving characters, and changing grass can make an old location appear visually new.

This produced one of the first unexpected behaviors in the project. The agent became fas-

minated by an area containing animated scenery. Small changes in the animation repeatedly triggered the novelty reward, so remaining in the same location became more rewarding than exploring the world. As the creator of the Youtube video interestingly pointed out, humans can exhibit similar behavior. We pursue short term rewards instead of doing productive things that perhaps pay off later, such as scrolling Tiktok or Instagram instead of studying or working. The agent was optimizing the specified reward correctly, but the reward did not perfectly represent the intended goal. The novelty criterion was adjusted so that small visual changes were no longer sufficient. The agent then began travelling farther through the map. This illustrates that exploration bonuses can be extremely useful, but their precise definition matters. A second important reward encouraged increases in the combined levels of the Pokémon in the agent's party. This gave the agent an incentive to participate in battles rather than running away from them. Over time, it learned to win battles, catch Pokémon, use alternative attacks, and evolve members of its party. Once again, the proxy reward produced unintended behavior. When the agent deposited a Pokémon into the storage system, the total level of its active party suddenly decreased. This produced a very large negative reward. After experiencing this event, the policy learned to avoid the entire Pokémon Center, even though Pokémon Centers are useful because they restore health. The reward was modified so that increases in levels produced positive reward without treating later decreases as equally large punishments. The agent then became willing to visit Pokémon Centers and learned to heal.

Later, the agent encountered a character selling a Magikarp. As the agent does not know that the laughably weak Magikarp eventually evolves into the powerful Gyarados, one would perhaps guess that the agent would pass on this offer. However, buying the Magikarp immediately increased the total levels of the party, even though the purchase was not particularly useful for progressing through the game. The agent therefore bought the Magikarp almost every time. It had discovered an easy way to maximize the proxy objective without achieving the broader intention behind that objective. These examples demonstrate a central difficulty in reinforcement learning. The agent optimizes the reward that is implemented, not the goal that the designer has in mind. A shaped reward may be written heuristically as

$$R_{t+1} = R_{t+1}^{\text{explore}} + R_{t+1}^{\text{level}} + R_{t+1}^{\text{progress}} + R_{t+1}^{\text{healing}} + \dots$$

Each term is intended to encourage useful behavior. The weights and definitions of the terms determine what behavior the agent actually learns. A poorly chosen proxy may create distraction, avoidance, stalling, or exploitation. For instance, when the agent lost a battle, which gave it a big negative reward as soon as the agent started the game up again, the agent would instead decide to indefinitely stay in the menu screen to avoid the negative reward. Naturally, the agent would never be able to complete the game this way which a human would recognize, but since the agent does not understand holistic goals such as "beat the game", it would rather wait in limbo forever than receiving a negative reward. Exemplifying why carefully defining rewards and proxies is so important.

5.0.3 Long horizons and emergent strategies

The difference between FrozenLake and Pokémon Red is not only the size of the observation space. Pokémon also has a much longer horizon. An action may influence events far into the future. Losing a battle may return the player to an earlier location. Training a Pokémon may make a later gym battle easier. Entering a new area may eventually unlock an important objective. The agent must therefore maximize a discounted return of the form

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

A discount factor close to one gives weight to consequences that occur many steps later. Even so, connecting a particular action to a distant reward remains difficult. Dense intermediate rewards make this credit-assignment problem more manageable, but they also introduce the reward-design problems described above. The trained policy developed navigation patterns that were not explicitly programmed. One observed strategy was a tendency to follow edges of the map in a consistent direction. Such a strategy can be useful when memory and planning are limited. Following the perimeter of an area eventually brings the agent past possible entrances and exits, even without an explicit map.

This is an example of emergent behavior. The developer did not write a rule telling the agent to follow walls. The behavior arose because policies with this tendency achieved higher returns during training. Emergent behavior can be useful, surprising, or undesirable. It is one of the reasons why reinforcement learning experiments often require extensive visualization and inspection rather than relying only on a numerical reward curve.

5.0.4 What the example teaches us

The Pokémon project contains the same formal ingredients as the simpler examples discussed earlier. There is an environment, a set of observations, a discrete action space, a reward signal, a stochastic policy, sampled trajectories, and an objective based on expected return. The difference is the scale and complexity of each ingredient.

FrozenLake allowed us to see the mathematical structure clearly. Pokémon Red shows what happens when that structure is placed inside a complicated environment. The state is only partially observed, exploration is difficult, rewards must be shaped, actions have long-term consequences, and the learned policy can exploit details that the designer did not anticipate.

The example also clarifies the meaning of model-free reinforcement learning. The agent interacts with a simulator but does not use an explicit transition model for planning. PPO learns a policy and value function directly from trajectories. A model-based method would instead learn or access a predictive model and use it to compare possible futures before acting.

Most importantly, the example shows that designing the reward is perhaps one of the most important parts of designing the problem. Reward shaping can make an otherwise impossible learning problem tractable, but every proxy reward changes the behavior that is encouraged. Watching the agent become distracted by animations, avoid a Pokémon Center, buy thousands of Magikarp, and eventually discover successful battle strategies illustrates this concept clearly.

Eventually the agent got stuck in a cave system, which proved to have too much of a monotone layout to trigger the exploration reward. This is where the developer stopped, hence the agent was never able to fully complete the game. We therefore challenge students to try and improve the code, and fully complete the game, though this may end up being a difficult task.